fiserv.

Technical Document

Datawire Secure Transport XML and SOAP Interfaces for Rapid Connect℠ Reference Guide

Document Version 1.28
February 2025

the speed of life®

Copyright Information

© 2025 Fiserv, Inc. or its affiliates. Fiserv is a registered trademark of Fiserv, Inc. All Rights Reserved. Privacy Policy

Printed in Canada.

Periodic revisions to this manual are issued to reflect technical changes and enhancements to the System, and to ensure that all information contained herein remains current and accurate.

Revision History

Date	Version	Description of Changes
4/30/2014	1.19	AuthKey2 becomes a fixed length zero padded (to the left) field of 8 alpha-numeric characters, instead of a fixed length zero padded field of 8 numeric characters in version 1.18.
5/27/2014	1.20	Updated firewall and IP information. Session management handling added for XML and SOAP transactions. Added Return Code 006. Updated SOAP interface sample codes.
7/22/2014	1.21	Updated caching information Added guidance as when to use session transactions Updated ClientRef description Updated StatusCode table
9/30/2016	1.22	Updated First Data logo
10/10/2019	1.23	Updated Copyright info and logo
3/11/2021	1.24	Updated StatusCode “Timeout” description in the Status Code table and added guidance for errors occuring mid session
2/18/2022	1.25	Updated to current Datawire brand
4/20/2022	1.26	Updated HTTP response headers, added error handling guidance for exceptions
3/14/2024	1.27	Corrected returned StatusCode to “WrongSessionContext” for transactions with invalid session context.
2/7/2025	1.28	Updated fonts as per latest Fiserv branding requirements

Table of Contents

Before You Begin 7

What are the XML and SOAP Interfaces for Rapid Connect? 7

Who Will Use These Interfaces? 7

System Requirements 7

Firewall and IP Information 7

HTTP Keep Alive Settings 8

SSL/TLS Session Reuse 8

HTTP Headers 8

Proxies and Dialup 8

Who to Contact if You Have Questions 8

The Datawire Secure Transport Network 9

What Is the Datawire Secure Transport Network? 9

Replacing Your Method of Transaction Transport 9

Interface and Method Summary 10

Before Sending Transactions 11

The XML Transaction 12

What Is a Datawire Secure Transport XML Transaction? 12

The Datawire Secure Transport XML Protocol 12

The XML Request Components 14

Component 1: The HTTP Request Header 14

Component 2: The XML Encoding and Version Details 15

Component 3: The XML Request Body 15

Sample XML Request with XML_Escape Encoding 21

Sample XML Request with Cdata Encoding 23

The XML Response Components 25

Component 1: HTTP Response Header 26

Component 2: The XML Encoding and Version Details 27

Component 3: The XML Response Body 27

Sample XML Response with XML_Escape Encoding 32

Sample XML Response with Cdata Encoding 36

Error Response Components 39
Component 1: HTTP Response Header 39
Component 2: The XML Encoding and Version Details 40
Component 3: The XML Response Body 40
Sample XML Error Response with a Return Code 42
XML_Escape Encoding Sample 42
Cdata Encoding Sample 46
Sample XML Error Response without a Return Code 49
The Soap Transaction 51
What Is a Datawire Secure Transport Soap Transaction? 51
The Datawire Secure Transport Soap Protocol 51
The Soap Request Components 53
Component 1: The HTTP Request Header 53
Component 2: The XML Encoding and Version Details 53
Component 3: The Soap Request Body 53
Sample Soap Request with XML_Escape Encoding 59
Sample Soap Request with Cdata Encoding 62
The Soap Response Components 65
Component 1: The HTTP Response Header 65
Component 2: The XML Encoding and Version Details 66
Component 3: The Soap Response Body 66
Sample Response with XML_Escape Encoding 72
Sample Response with Cdata Encoding 75
SOAP Error Response Components 79
Component 1: The HTTP Response Header 79
Component 2: The XML Encoding and Version Details 80
Component 3: The SOAP Response Body 80
Sample SOAP Error Response with XML_ Escape Encoding 83
Sample SOAP Error Response with Cdata Encoding 86
Sample SOAP Error Response without a Return Code 90
Error Management 92
Error Handling 92
Error Types 92

HTTP Errors & Handling 92
Exceptions 93
Return Code Values 93
Status Code Values 95
SOAP Interface Code Samples 96
C# Sample 96
VB Net Sample 100
JAVA Sample 104
PHP Sample 108

Before You Begin

In this Chapter

- What are the XML and SOAP Interfaces for Rapid Connect?
- Who will use these Interfaces?
- System Requirements
- Firewall and IP Information
- Keep Alive Settings
- SSL Session Reuse
- Proxies and Dialup
- Who to Contact if You Have Questions?

What are the XML and SOAP Interfaces for Rapid Connect?

The Datawire Secure Transport XML and SOAP Interfaces for Rapid Connect provide a means by which a POS application can utilize the Datawire Secure Transport Network to deliver transactions to their corresponding payment processor through the Rapid Connect platform.

Transactions are formatted and sent from the POS application through the Datawire Secure Transport Network to the Rapid Connect platform where these transactions are routed to their corresponding processor(s). The Datawire Secure Transport Network is a transparent transaction processing network that does not store or manage credit card credentials. No mechanism is available to store and retrieve previously sent cardholder data within the Datawire Secure Transport Network transaction processing stream. The vendor or merchant is responsible for storing and managing this data.
For more information, refer to the "What is the Datawire Secure Transport Network?" section.

Who Will Use These Interfaces?

Both XML and SOAP Interfaces will be offered to any vendor/merchant who needs a mechanism to send transactions to Rapid Connect through the Datawire Secure Transport Network. A vendor/merchant can choose either to use an XML or a SOAP framework.

System Requirements

To implement the Datawire Secure Transport XML Interface, you can use any environment that supports HTTP over SSL/TLS.

To implement the Datawire Secure Transport SOAP Interface, you can use any SOAP Framework/Tool that is SOAP 1.1 compliant.

Firewall and IP Information

Datawire Secure Transport recommends that firewalls be employed and situated between the internal corporate network on which the Datawire Secure Transport-compliant devices and applications reside and the Internet at large.

Datawire Secure Transport-compliant devices and applications require only **outbound** access through the corporate firewall to the Datawire Secure Transport Edge Nodes.

Datawire Secure Transport strongly recommends that outbound firewalls not restrict Datawire Secure Transport-

compliant devices and applications to specific destination Edge Node IP addresses. The outbound destination TCP port 443 (for HTTPS integrated Devices/Applications) has been defined for use by all of the Datawire Secure Transport-compliant devices and application's communication with the Datawire Secure Transport Edge Nodes.

Datawire Secure Transport-compliant devices and applications require **DNS** (Domain Name Server) to find the IP addresses of the Datawire Secure Transport Edge Nodes to which POS transactions will be sent. Caching the IP addresses of Datawire Secure Transport servers is strongly discouraged for non-session transactions. Therefore, the DNS service should be used periodically to determine the destination IP address of Datawire Secure Transport servers.

For session transactions, it is recommended that client applications using this interface do not refresh DNS cache until one full session is completed. This only applies in cases where multiple TCP connections are to be used between the client application and Datawire Secure Transport's transaction URLs. This is to avoid invalidating a session as a result of a DNS refresh.

HTTP Keep Alive Settings

HTTP 1.1 supports a **keep-alive** option allowing a client to continue using an existing connection for new Requests and avoiding the TCP connection time. If you wish to use this option, you need to ensure your HTTP libraries support keep alive. Most libraries, such as JDK (from 1.4), support both of these features.

SSL/TLS Session Reuse

The SSL handshake is a time consuming task. Most SSL libraries support SSL session continuation, allowing a client to reuse a previously negotiated key within a set period of time. You need to ensure your SSL and HTTP libraries support session continuation. Most libraries, such as JDK (from 1.4), support both of these features.

HTTP Headers

The HTTP message-header fields allow the client and the server to pass additional information with an HTTP request or response. Header fields are colon-separated key-value pairs in clear text format.

As per HTTP RFC standards, the line terminator for message-header fields is the sequence CRLF (\r\n). The last header is separated by an additional CRLF from the following message body. (In other words, blank line between headers and message body is required.) Please refer to RFC 7230 for additional information.

Proxies and Dialup

The scope of this document is limited to the XML and SOAP Interfaces and its use for integrating access to the Datawire Secure Transport Network. It does not cover such items as deployment behind proxies and dial backup. Many operating environments and libraries offer transparent support for running HTTPS applications behind a **proxy server** — it is the responsibility of the application developer to ensure that a POS application supports any special network requirements. For instance, Java (from 1.4) supports both HTTPS and SOCKS proxies.

In the event that an Internet connection becomes unavailable, it would be advisable to be prepared to provide **dial backup capability**. Dial backup can be achieved by obtaining backup Internet access via dialup (many network routers supports dial backup to an ISP).

Who to Contact if You Have Questions

Should you have any further questions **after reviewing this Reference Guide**, please contact the Datawire Secure Transport Vendor Integration (VIP) Team at securetransport.integration@fiserv.com

The Datawire Secure Transport Network

In This Chapter

- What is the Datawire Secure Transport Network?
- Replacing Your Current Method of Transaction Transport

What Is the Datawire Secure Transport Network?

The Datawire Secure Transport Network is a transaction transport network that moves transactions from point-of-sale applications and devices through the Datawire Secure Transport Network to the Rapid Connect platform to the respective payment processor and back.

The Datawire Secure Transport Network has been architected and built from the ground up to provide secure, reliable and rapid transaction transport. The Datawire Secure Transport Network is unique in its ability to offer consistently strong performance independent of the condition of the underlying Internet infrastructure.

Replacing Your Method of Transaction Transport

These interfaces provide a means by which a Point of Sale (POS) application can wrap and send Rapid Connect messages securely through the Datawire Secure Transport Network over the Internet to the Rapid Connect platform to the respective payment processor and back to the POS application. The Datawire Secure Transport Network merely replaces your current method of transaction transport.

Note: Please refer to your Rapid Connect-specific documentation for additional requirements.

To Send a Transaction using the Datawire Secure Transport Network:

1. The **POS application** wraps the transaction request in an XML or SOAP document.
2. The Request is sent through the **Internet** to the Datawire Secure Transport Network **Servers**.
3. The Datawire Secure Transport Network **Servers** send the Request to the Rapid Connect Platform.
4. The Rapid Connect Platform processes the Request and passes it along to the Payment Processor.
5. The Payment Processor returns the Response to the Rapid Connect Platform.
6. The Rapid Connect Platform sends the Response to the Datawire Secure Transport Network **Servers**.
7. The Datawire Secure Transport Network **Servers** send the Response through the Internet to the **POS application**.
8. The **POS application** extracts the Response, completing the transaction.

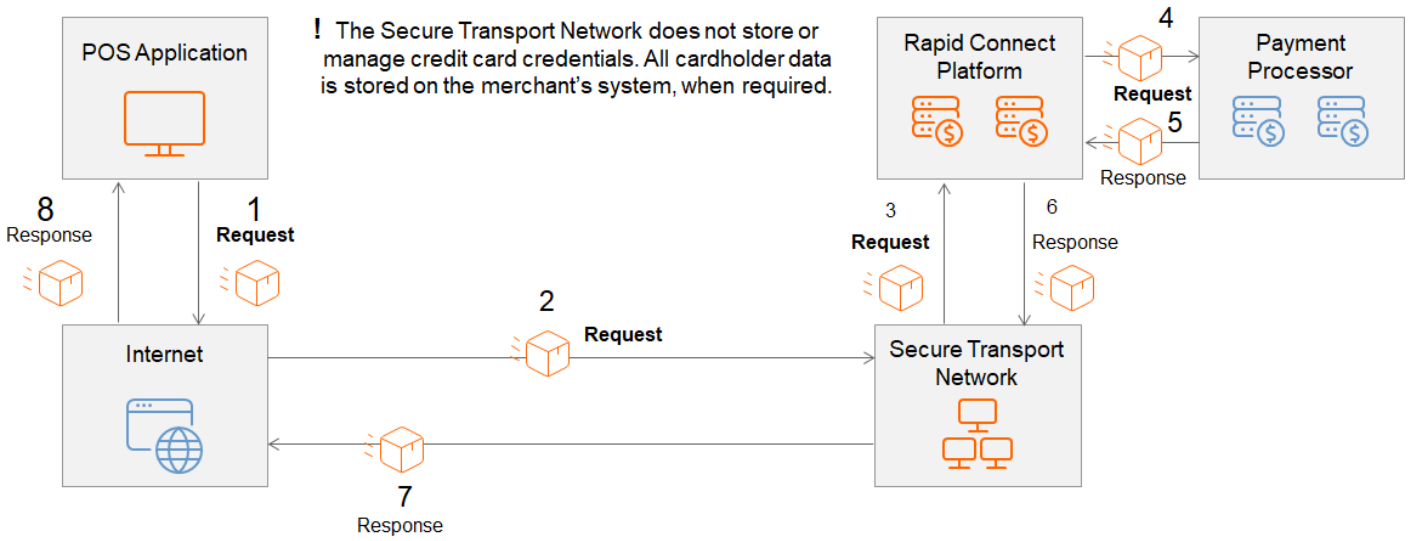

Image 1: Replacing Your Method of Transaction Transport

Interface and Method Summary

There are two supported interfaces: XML and SOAP over HTTP. For detailed information about the XML Interface, refer to "The XML Transaction" chapter. For detailed information about the SOAP Interface, refer to "The SOAP Transaction" chapter.

The **HTTP POST** method is the only supported method. The **GET** method is not supported.

Before Sending Transactions

Before a merchant can send transactions through the Datawire Secure Transport Network, the vendor/merchant will need at least:

- A URL to which transactions will be sent.
- A **ServiceID** to identify the Rapid Connect platform to which the tranactions will be sent.
- A Datawire Secure Transport ID (**DID**) to authenticate each transaction.

The detailed steps and information required for vendors to be provisioned on the Datawire Secure Transport Network will be provided by the Datawire Secure Transport Vendor Integration Team (securetransport.integration@fiserv.com).

The XML Transaction

In This Chapter

- What is a Datawire Secure Transport XML Transaction?
- The Datawire Secure Transport XML Protocol
- Sending XML Requests and Receiving XML Responses
- The XML Request Components
- Sample XML Request with XML-Encoding
- Sample XML Request with CData Encoding
- The XML Response Components
- Sample XML Response with XML-Encoding
- Sample XML Response with CData Encoding
- XML Error Response Components
- Sample XML Error Response with Return Code
- Sample XML Error Response without Return Code

What Is a Datawire Secure Transport XML Transaction?

A Datawire Secure Transport XML transaction contains all the transaction data necessary to process your transaction
at the Rapid Connect platform. The Datawire Secure Transport Network simply replaces your current method of processing transactions.

Note: As the transaction passes through the Datawire Secure Transport Network, this transaction data is never stored on the Network for future use. The transaction data, if needed, must be stored on the merchant's system so that it can be used for transactions such as reversals and settlements.

The Datawire Secure Transport XML Protocol

The Datawire Secure Transport XML Protocol is composed of a request-response pair. Request and Response messages are sent using HTTP. Each Request message is an XML document with a Request element as a document root and is sent as an HTTP POST. The transaction is sent to the Datawire Secure Transport Network Servers (Edge Nodes) that accept the HTTPS (HTTP over SSL) request. The Datawire Secure Transport Network Servers (Edge Nodes) will then route the transaction through the Datawire Secure Transport Network to the Rapid Connect platform, and receive and route the appropriate Response back to the POS application.

A Simple Transaction is composed of a single request-response pair. Payment transactions that have a "simple" nature should be wrapped up and sent as Simple Transactions. A typical example of a Simple Transaction is a credit card or debit card authorization, reversal, completion, etc...

A Session Transaction is composed of multiple request-response pairs that must be delivered to the payment processing server in sequential order. Session transactions should only be used when a simple request/response type of interaction cannot fulfill the end- to-end processing case, e.g. there are lots of level 3 line items that need to be carried over along with a transaction.

To Send and Receive Simple Transactions:

1. The POS application wraps the **Transaction** Request in an XML document.
2. The POS application posts the XML document over HTTPS to the Datawire Secure Transport Network.

   Once the **Transaction** Request is received, the Datawire Secure Transport Network will route the Request to the Rapid Connect platform (the one specified by the **ServiceID**).
3. The Rapid Connect platform returns the Response in its own format. The Datawire Secure Transport Network translates this Response into the appropriate XML document.

   If the transaction is successful, the **ReturnCode** is “000” and the results of the transaction are returned to the POS application in the **TransactionResponse** sub-element of the **Response** element.

   However, if the transaction is unsuccessful, the **ReturnCode** and **StatusCode** can vary and you need to be prepared to manage the error. For more information, refer to the “Return Code Values” section and the “Status Code Values” section.
4. The POS application then closes the HTTPS connection if there are no additional transactions to be sent.

To Send and Receive Session Tranasctions:

1. The POS application wraps an **InitiateSession** Request in an XML document.
2. The POS application posts the XML document over HTTPS to the Datawire Secure Transport Network.

   Once the **InitiateSession** Request is received, the Datawire Secure Transport Network will establish a connection to and route the payload to the Rapid Connect platform (the one specified by the **ServiceID**).
3. The Rapid Connect platform returns the Response in its own format. The Datawire Secure Transport Network translates this Response into the appropriate XML document.

   If the transaction is successful, the **ReturnCode** is "000" and the results of the transaction are returned to the POS application in the **InitiateSessionResponse** sub-element of the **Response** element. The value of **SessionContext** element (a sub- element of InitiateSessionResponse) should be saved by POS application and will be used as is in subsequent session transactions.

   However, if the transaction is unsuccessful, the **ReturnCode** and **StatusCode** can vary and you need to be prepared to manage the error. For more information, refer to the "Return Code Values" section as well as the "Status Code Values" section.
4. The POS application wraps a **SessionTransaction** Request with the **SessionContext** it received in the **InitiateSessionResponse** in an XML document.
5. The POS application posts the XML document over HTTPS to the Datawire Secure Transport Network.

   Once the **SessionTransaction** Request is received, the Datawire Secure Transport Network will route the payload to the Rapid Connect platform (through the same route of the **InitiateSession** transaction).
6. The Rapid Connect platform returns the Response in its own format. The Datawire Secure Transport Network translates this Response into the appropriate XML document.

   If the transaction is successful, the **ReturnCode** is "000" and the results of the transaction are returned to the POS application in the **SessionTransactionResponse** sub-element of the **Response** element.

   However, if the transaction is unsuccessful, the **ReturnCode** and **StatusCode** can vary and you need to be prepared to manage the error.
7. Repeat step 4 to 6 until the POS application is ready to end this session.
8. The POS application wraps a **TerminateSession** Request with the **SessionContext** it received in the

InitiateSessionResponse in an XML document.

9. The POS application posts the XML document over HTTPS to the Datawire Secure Transport Network.

   Once the **TerminateSession** Request is received, the Datawire Secure Transport Network will route the payload to the Rapid Connect platform (through the same route of the **InitiateSession** transaction).

10. The Rapid Connect platform returns the Response in its own format. The Datawire Secure Transport Network translates this Response into the appropriate XML document.

    If the transaction is successful, the **ReturnCode** is "000" and the results of the transaction are returned to the POS application in the **TerminateSessionResponse** sub-element of the Response element.

    However, if the transaction is unsuccessful, the **ReturnCode** and **StatusCode** can vary and you need to be prepared to manage the error.

11. Datawire Secure Transport Network closes the connection to the Rapid Connect platform.

The XML Request Components

The XML Request is made up of the three major components below. For more information regarding the XML components, refer to the XML schema file. In addition, for a sample, refer to the samples at the end of this section.

Component 1: The HTTP Request Header

The first component of the Request is the HTTP Request Header, which includes the minimum required fields described below.

Parameters

User-Agent	The User-Agent element identifies the application that has generated and is sending the transaction. Contact the Datawire Secure Transport Vendor Integration Team (securetransport.integration@fiserv.com) for more information.
Host	The Host parameter is the name of the server to which the request is sent, and it is derived from the URL. Specifically, if the request is sent to the URL https://abc.123.net/path/to/dwxml, then the contents of this field must be “abc.123.net”. This parameter will be provided by securetransport.integration@fiserv.com and is required in order for the Request to be successful.
Connection	To optimize transaction speed, you may use Keep-alive. For more information, refer to the “HTTP Keep Alive Settings” section.
Cache-Control	Cache must be turned off because each time you send a new request, the application needs to receive a fresh response from the Datawire Secure Transport Network Servers (Edge Nodes). Applications cannot receive cached responses.
Content-Length	Specifies the exact size of the request body, in bytes.
Content-Type	Defines the MIME type for the message and the character encoding (optional) used for the XML body of the request. The Content-Type must be text/xml or the message will not be accepted by the Datawire Secure Transport Network.

Component 2: The XML Encoding and Version Details

The second component of the Request contains the XML encoding and version details as sampled here:

```
<?xml version="1.0" encoding="UTF-8"?>
```

All text should comply with the standard XML character encoding rules as described in the XML specification at w3c.org. Please refer to the section outlined below: https://www.w3.org/TR/xml/#charencoding

Component 3: The XML Request Body

The third component of the request is the **Request** root element, which includes the **Version, ClientTimeout**, and XML **NameSpace** attributes and its sub-elements as detailed below.

Version	<Request Version = "3"> is mandatory for each request as it defines the current version of the Datawire Secure Transport XML Interface. Any future versions of the Interface will have different versions for this attribute.
ClientTimeout	The ClientTimeout attribute holds the number of seconds before the client will timeout due to inactivity.
xmlns or prefixed with xmlns:	The XML Namespace attribute is as follows: xmlns=http://securetransport.dw/rcservice/xml

The Sub-Elements

The **Request** root includes the **ReqClientID** and **Transaction** sub-elements as detailed below.

The ReqClientID Sub-Element

The **ReqClientID** sub-element is included in every request to identify and authenticate each request being sent through the Datawire Secure Transport Network. The **ReqClientID** sub-element is described below:

Parameters

DID	The DID (Datawire Secure Transport ID) is a unique customer identifier generated by the Datawire Secure Transport and assigned to each client using the Datawire Secure Transport Network. This DID is obtained from the Datawire Secure Transport Vendor Integration Team (securetransport.integration@fiserv.com) or by the SRS process. The maximum DID length is 32 characters. The value of the DID element in the response echoes what appears in the request.
App	The App element identifies the application that has generated and is sending the transaction. This is a 15 char alphanumeric code that identifies each application and is provided by the Datawire Secure Transport Vendor Integration Team (securetransport.integration@fiserv.com).

Auth	The Auth element specifies the authentication information needed to process the transaction: AuthKey1 and AuthKey2. The Auth element format is AuthKey1 followed by "|" followed by AuthKey2 (AuthKey1|AuthKey2). RapidConnect Support maintains the authority of the ultimate explanation on how to populate AuthKey1 and AuthKey2 from RapidConnect system parameters, below is just a brief illustration about the content of AuthKey1 and AuthKey2: -- AuthKey1 is a field of up to 32 alpha-numeric characters. -- AuthKey2 is a fixed length zero padded (to the left) field of 8 alpha-numeric characters. -- AuthKey1 is composed of Group ID followed by Merchant ID. Group ID and Merchant ID are parameters assigned by the Processor/RapidConnect system, please contact RapidConnect Support for specific Group ID and Merchant ID requirements. -- AuthKey2 is composed of Terminal ID (Front End TID). Terminal ID is a parameter assigned by the Processor/RapidConnect system, please contact RapidConnect Support for theTerminal ID. -- As an example, when given RapidConnect system parameters: Group ID=10001, Merchant ID=1054217, Terminal ID=1032339, AuthKey1 should be populated as "100011054217", AuthKey2 should be populated as "01032339", the final Auth is "100011054217|01032339".
ClientRef	The ClientRef element identifies the packet sent by the application. This element needs to be unique to identify each transaction separately. A transaction counter is an acceptable method of creating this unique transaction identifier. For more information, contact securetransport.integration@fiserv.com

The Transaction Sub-Element

The second sub-element of the "Request" root element is the **Transaction** sub-element, which contains the ServiceID, and the **Payload** sub-elements as shown in the sample at the end of the chapter.

Parameters

ServiceID	The ServiceID sub-element is an identifier that is used to identify the payment processor to which the transactions will be sent for all subsequent transactions. The ServiceID is provided by the Datawire Secure Transport VIP Team (securetransport.integration@fiserv.com).
Payload	Note: If the encoding attribute is not specified, the default encoding is **xml_escape**. The raw format of the Rapid Connect message is a standard xml message. The encoded format of the Rapid Connect xml message is carried in the Datawire Secure Transport Payload element. The encoding is specified by the Payload's encoding attribute that could be one of two values: **xml_escape** or **cdata**. Any other encodings are not supported. **xml_escape:** <Payload Encoding="xml_escape">xml_escaped_RapidConnect_Message</Payload> **where** xml_escaped_RapidConnect_Message = xml_escape(raw_RapidConnect_XML_message) **cdata**: For **cdata** encoding, the intact raw Rapid Connect XML message is wrapped inside the <![**CDATA**[and]]>. <Payload Encoding="**cdata**"><![**CDATA**[raw_RapidConnect_XML_message]]></Payload> **Important Encoding Details** You must only apply the encoding once for the Payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly. The value of encoding attribute in a response echoes the encoding value of its corresponding request. For xml_escape encoding, any occurances of the character "<", ">" or "&" in the raw Rapid Connect XML message will be replaced by "<", ">" and "&" respectively. For cdata encoding: Be cautious: the raw Rapid Connect XML message CANNOT contain "]]>". Use xml_escape encoding if this sequence of characters appear in the raw Rapid Connect XML message. If the merchant application chooses cdata encoding, the Datawire Secure Transport Network will apply this encoding as well in its response – in this process Datawire Secure Transport Network will NOT check if the raw Rapid Connect XML response message contains "]]>". **Examples** In the example below, you can see how the raw XML data is encoded differently: Raw XML message payload: "<RC>Hello, A & B</RC>" **xml_escape** encoded message payload: "<RC>Hello,A & B</RC> " cdata encoded message payload: "<![CDATA[<RC>Hello,A & B</RC>]]>"

The InitiateSession Sub-Element

For an InitiateSession request, the second sub-element of the "Request" root element is the **InitiateSession** sub-element, which contains the ServiceID, and the **Payload** sub-elements. The Payload sub-element is optional.

Parameters

ServiceID	The ServiceID sub-element is an identifier that is used to identify the payment processor to which the transactions will be sent for all subsequent transactions. The ServiceID is provided by the Datawire Secure Transport VIP Team (securetransport.integration@fiserv.com).
Payload	**Note**: If the encoding attribute is not specified, the default encoding is **xml_escape**. The raw format of the Rapid Connect message is a standard xml message. The encoded format of the Rapid Connect xml message is carried in the Datawire Secure Transport Payload element. The encoding is specified by the Payload's encoding attribute, which could be one of two values: **xml_escape** or **cdata**. Any other encodings are not supported. **xml_escape:** <Payload Encoding="xml_escape">xml_escaped_RapidConnect_Message</Payload> **where** xml_escaped_RapidConnect_Message = xml_escape(raw_RapidConnect_XML_message) **cdata**: For **cdata** encoding, the intact raw Rapid Connect XML message is wrapped inside the <![**CDATA**[and]]>. <Payload Encoding="**cdata**"><![**CDATA**[raw_RapidConnect_XML_message]]></Payload> **Important Encoding Details** You must only apply the encoding once for the Payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly. The value of encoding attribute in a response echoes the encoding value of its corresponding request. For xml_escape encoding, any occurances of the character "<", ">" or "&" in the raw Rapid Connect XML message will be replaced by "<", ">" and "&" respectively. For cdata encoding: Be cautious: the raw Rapid Connect XML message CANNOT contain "]]>". Use xml_escape encoding if this sequence of characters appear in the raw Rapid Connect XML message. If the merchant application chooses cdata encoding, the Datawire Secure Transport Network will apply this encoding as well in its response – in this process Datawire Secure Transport Network will NOT check if the raw Rapid Connect XML response message contains "]]>". **Examples** In the example below, you can see how the raw XML data is encoded differently: Raw XML message payload: "<RC>Hello, A & B</RC>" **xml_escape** encoded message payload: "<RC>Hello,A & B</RC> " cdata encoded message payload: "<![CDATA[<RC>Hello,A & B</RC>]]>"

The SessionTransaction Sub-Element

For a SessionTransaction request, the second sub-element of the “Request” root element is the **SessionTransaction** sub-element, which contains the SessionContext, and the **Payload** sub-elements.

Parameters

SessionContext	The SessionContext is what has been returned in the response of the previous InitiateSession request. It guarantees that all the transactions in the session will go through the same route to the payment processor.
Payload	Note: If the encoding attribute is not specified, the default encoding is **xml_escape**. The raw format of the Rapid Connect message is a standard xml message. The encoded format of the Rapid Connect xml message is carried in the Datawire Secure Transport Payload element. The encoding is specified by the Payload’s encoding attribute, which could be one of two values: **xml_escape** or **cdata**. Any other encodings are not supported. **xml_escape:** <Payload Encoding="xml_escape">xml_escaped_RapidConnect_Message</Payload> **where** xml_escaped_RapidConnect_Message = xml_escape(raw_RapidConnect_XML_message) **cdata**: For **cdata** encoding, the intact raw Rapid Connect XML message is wrapped inside the <![**CDATA**[and]]>. <Payload Encoding="**cdata**"><![**CDATA**[raw_RapidConnect_XML_message]]></Payload> **Important Encoding Details** You must only apply the encoding once for the Payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly. The value of encoding attribute in a response echoes the encoding value of its corresponding request. For xml_escape encoding, any occurances of the character "<",">" or "&" in the raw Rapid Connect XML message will be replaced by "<", ">" and "&" respectively. For cdata encoding: Be cautious: the raw Rapid Connect XML message CANNOT contain “]]>”. Use xml_escape encoding if this sequence of characters appear in the raw Rapid Connect XML message. If the merchant application chooses cdata encoding, the Datawire Secure Transport Network will apply this encoding as well in its response – in this process Datawire Secure Transport Network will NOT check if the raw Rapid Connect XML response message contains "]]>". **Examples** In the example below, you can see how the raw XML data is encoded differently: Raw XML message payload: "<RC>Hello, A & B</RC>" **xml_escape** encoded message payload: "<RC>Hello,A & B</RC> " cdata encoded message payload: "<![CDATA[<RC>Hello,A & B</RC>]]>"

The TerminateSession Sub-Element

For a TerminateSession request, the second sub-element of the “Request” root element is the **TerminateSession** sub-element, which contains the SessionContext, and the **Payload** sub-elements. The Payload sub-element is optional.

Parameters

SessionContext	The SessionContext is what has been returned in the response of the previous InitiateSession request. It guarantees that all the transactions in the session will go through the same route to the payment processor.
Payload	Note: If the encoding attribute is not specified, the default encoding is **xml_escape**. The raw format of the Rapid Connect message is a standard xml message. The encoded format of the Rapid Connect xml message is carried in the Datawire Secure Transport Payload element. The encoding is specified by the Payload’s encoding attribute, which could be one of two values: **xml_escape** or **cdata**. Any other encodings are not supported. **xml_escape:** <Payload Encoding="xml_escape">xml_escaped_RapidConnect_Message</Payload> **where** xml_escaped_RapidConnect_Message = xml_escape(raw_RapidConnect_XML_message) **cdata**: For **cdata** encoding, the intact raw Rapid Connect XML message is wrapped inside the <![**CDATA**[and]]>. <Payload Encoding="**cdata**"><![**CDATA**[raw_RapidConnect_XML_message]]></Payload> **Important Encoding Details** You must only apply the encoding once for the Payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly. The value of encoding attribute in a response echoes the encoding value of its corresponding request. For xml_escape encoding, any occurances of the character "<", ">" or "&" in the raw Rapid Connect XML message will be replaced by "<", ">" and "&" respectively. For cdata encoding: Be cautious: the raw Rapid Connect XML message CANNOT contain “]]>”. Use xml_escape encoding if this sequence of characters appear in the raw Rapid Connect XML message. If the merchant application chooses cdata encoding, the Datawire Secure Transport Network will apply this encoding as well in its response – in this process Datawire Secure Transport Network will NOT check if the raw Rapid Connect XML response message contains "]]>". **Examples** In the example below, you can see how the raw XML data is encoded differently: Raw XML message payload: "<RC>Hello, A & B</RC>" **xml_escape** encoded message payload: "<RC>Hello,A & B</RC> " cdata encoded message payload: "<![CDATA[<RC>Hello,A & B</RC>]]>"

Sample XML Request with XML_Escape Encoding

The sample below is an example of what a Request with **xml_escape** encoding may look like.
Note: The sample below is only provided for your reference and thus contains only sample values.

```
POST /xyz HTTP/1.1
User-Agent: AppName_VersionNumber
Host: Host Portion of URL
Connection: Keep-Alive
Cache-Control: no-cache
Content-Length: length_of_body (actual length of HTTP message body)
Content-Type: text/xml

<?xml version="1.0" encoding="UTF-8"?>
<Request Version="3" ClientTimeout="Sample_TimeoutValue" xmlns="http://securetransport.dw/rcservice/xml">
    <ReqClientID>
        <DID>Sample_DID</DID>
        <App>Sample_AppName</App>
        <Auth>Sample_AuthKey1|Sample_AuthKey2</Auth>
        <ClientRef>Sample_ClientRef</ClientRef>
    </ReqClientID>
    <Transaction>
        <ServiceID>Sample_ServiceID</ServiceID>
        <Payload Encoding="xml_escape">&lt;RC&gt;Hello, A &amp; B&lt;/RC&gt;</Payload>
    </Transaction>
</Request>
```

Note: The sample below of InitiateSession is only provided for your reference and thus contains only sample values.

```
POST /xyz HTTP/1.1
User-Agent: AppName_VersionNumber
Host: Host Portion of URL
Connection: Keep-Alive
Cache-Control: no-cache
Content-Length: length_of_body (actual length of HTTP message body)
Content-Type: text/xml

<?xml version="1.0" encoding="UTF-8"?>
<Request Version="3" ClientTimeout="Sample_TimeoutValue" xmlns="http://securetransport.dw/rcservice/xml">
    <ReqClientID>
        <DID>Sample_DID</DID>
```

```
        <App>Sample_AppName</App>
        <Auth>Sample_AuthKey1|Sample_AuthKey2</Auth>
        <ClientRef>Sample_ClientRef</ClientRef>
    </ReqClientID>
    <InitiateSession>
        <ServiceID>Sample_ServiceID</ServiceID>
        <Payload Encoding="xml_escape">&lt;RC&gt;Hello, A &amp; B&lt;/RC&gt;</Payload>
    </InitiateSession>
</Request>
```

Note: The sample below of SessionTransaction is only provided for your reference and thus contains only sample values.

```
POST /xyz HTTP/1.1
User-Agent: AppName_VersionNumber
Host: Host Portion of URL
Connection: Keep-Alive
Cache-Control: no-cache
Content-Length: length_of_body (actual length of HTTP message body)
Content-Type: text/xml

<?xml version="1.0" encoding="UTF-8"?>
<Request Version="3" ClientTimeout="Sample_TimeoutValue" xmlns="http://securetransport.dw/rcservice/xml">
    <ReqClientID>
        <DID>Sample_DID</DID>
        <App>Sample_AppName</App>
        <Auth>Sample_AuthKey1|Sample_AuthKey2</Auth>
        <ClientRef>Sample_ClientRef</ClientRef>
    </ReqClientID>
    <SessionTransaction>
        <SessionContext>105|19388|15731|233</SessionContext>
        <Payload Encoding="xml_escape">&lt;RC&gt;Hello, A &amp; B&lt;/RC&gt;</Payload>
    </SessionTransaction>
</Request>
```

Note: The sample below of TerminateSession is only provided for your reference and thus contains only sample values.

```
POST /xyz HTTP/1.1
User-Agent: AppName_VersionNumber
Host: Host Portion of URL
Connection: Keep-Alive
```

```
Cache-Control: no-cache
Content-Length: length_of_body (actual length of HTTP message body)
Content-Type: text/xml

<?xml version="1.0" encoding="UTF-8"?>
<Request Version="3" ClientTimeout="Sample_TimeoutValue" xmlns="http://securetransport.dw/rcservice/xml">
    <ReqClientID>
        <DID>Sample_DID</DID>
        <App>Sample_AppName</App>
        <Auth>Sample_AuthKey1|Sample_AuthKey2</Auth>
        <ClientRef>Sample_ClientRef</ClientRef>
    </ReqClientID>
    <TerminateSession>
        <SessionContext>105|19388|15731|233</SessionContext>
        <Payload Encoding="xml_escape">&lt;RC&gt;Hello, A &amp; B&lt;/RC&gt;</Payload>
    </TerminateSession>
</Request>
```

Sample XML Request with Cdata Encoding

Note: The sample below is only provided for your reference and thus contains only sample values.

```
POST /xyz HTTP/1.1
User-Agent: AppName_VersionNumber
Host: Host Portion of URL
Connection: Keep-Alive
Cache-Control: no-cache
Content-Length: length_of_body (actual length of HTTP message body)
Content-Type: text/xml

<?xml version="1.0" encoding="UTF-8"?>
<Request Version="3" ClientTimeout="Sample_TimeoutValue" xmlns="http://securetransport.dw/rcservice/xml">
    <ReqClientID>
        <DID>Sample_DID</DID>
        <App>Sample_AppName</App>
        <Auth>Sample_AuthKey1|Sample_AuthKey2</Auth>
        <ClientRef>Sample_ClientRef</ClientRef>
    </ReqClientID>
    <Transaction>
        <ServiceID>Sample_ServiceID</ServiceID>
        <Payload Encoding="cdata"><![CDATA[<RC>Hello, A & B</RC>]]></Payload>
```

```
    </Transaction>
</Request>
```

Note: The sample below of InitiateSession is only provided for your reference and thus contains only sample values.

```
POST /xyz HTTP/1.1
User-Agent: AppName_VersionNumber
Host: Host Portion of URL
Connection: Keep-Alive
Cache-Control: no-cache
Content-Length: length_of_body (actual length of HTTP message body)
Content-Type: text/xml

<?xml version="1.0" encoding="UTF-8"?>
<Request Version="3" ClientTimeout="Sample_TimeoutValue" xmlns="http://securetransport.dw/rcservice/xml">
    <ReqClientID>
        <DID>Sample_DID</DID>
        <App>Sample_AppName</App>
        <Auth>Sample_AuthKey1|Sample_AuthKey2</Auth>
        <ClientRef>Sample_ClientRef</ClientRef>
    </ReqClientID>
    <InitiateSession>
        <ServiceID>Sample_ServiceID</ServiceID>
        <Payload Encoding="cdata"><![CDATA[<RC>Hello, A & B</RC>]]></Payload>
    </InitiateSession>
</Request>
```

Note: The sample below of SessionTransaction is only provided for your reference and thus contains only sample values.

```
POST /xyz HTTP/1.1
User-Agent: AppName_VersionNumber
Host: Host Portion of URL
Connection: Keep-Alive
Cache-Control: no-cache
Content-Length: length_of_body (actual length of HTTP message body)
Content-Type: text/xml

<?xml version="1.0" encoding="UTF-8"?>
<Request Version="3" ClientTimeout="Sample_TimeoutValue" xmlns="http://securetransport.dw/rcservice/xml">
```

```
    <ReqClientID>
        <DID>Sample_DID</DID>
        <App>Sample_AppName</App>
        <Auth>Sample_AuthKey1|Sample_AuthKey2</Auth>
        <ClientRef>Sample_ClientRef</ClientRef>
    </ReqClientID>
    <SessionTransaction>
        <SessionContext>105|19388|15731|233</SessionContext>
        <Payload Encoding="cdata"><![CDATA[<RC>Hello, A & B</RC>]]></Payload>
    </SessionTransaction>
</Request>
```

Note: The sample below of TerminateSession is only provided for your reference and thus contains only sample values.

```
POST /xyz HTTP/1.1
User-Agent: AppName_VersionNumber
Host: Host Portion of URL
Connection: Keep-Alive
Cache-Control: no-cache
Content-Length: length_of_body (actual length of HTTP message body)
Content-Type: text/xml

<?xml version="1.0" encoding="UTF-8"?>
<Request Version="3" ClientTimeout="Sample_TimeoutValue" xmlns="http://securetransport.dw/rcservice/xml">
    <ReqClientID>
        <DID>Sample_DID</DID>
        <App>Sample_AppName</App>
        <Auth>Sample_AuthKey1|Sample_AuthKey2</Auth>
        <ClientRef>Sample_ClientRef</ClientRef>
    </ReqClientID>
    <TerminateSession>
        <SessionContext>105|19388|15731|233</SessionContext>
        <Payload Encoding="cdata"><![CDATA[<RC>Hello, A & B</RC>]]></Payload>
    </TerminateSession>
</Request>
```

The XML Response Components

A successful response contains the three major components listed below. For more information regarding the XML components, refer to the XML schema file. In addition, for a sample, refer to the samples at the end of this

section.

Component 1: HTTP Response Header

The Secure Transport HTTP response headers are driven by industry standards and PCI regulatory requirements.

Datawire Secure Transport Network may add new HTTP response header(s), delete some HTTP response headers, or change values of some of the HTTP response headers. This may be required to comply with security and/or industry best practices. Any API user should bear this in mind and make its application conform to respective HTTP RFCs to allow robustness without affecting its customers.

The HTTP Response Headers included below are provided as samples and are subject to change based on industry requirements.

Parameters

Date	The day, month, year, 00:00:00 GMT of the Response.
Server	The Server returning the Response.
Content-Length	Specifies the exact size of the response body, in bytes.
Content-Type	The Content-Type must be text/xml or the Secure Transport Network will not accept the message.
Cache-Control	The Cache-Control header directives specify behavior intended to prevent caches from adversely interfering with the request or response. These directives typically override the default caching algorithms.
X-XSS-Protection	The X-XSS-Protection response header helps prevent reflected cross-site scripting attacks.
X-Content-Type-Options	The X-Content-Type-Options response header is a marker used by the server to indicate that the MIME types advertised in the Content-Type headers should not be changed and be followed.
Strict-Transport-Security	The Strict-Transport-Security response header allows for a more effective implementation of TLS by ensuring all communication takes place over a secure transport layer on the client side.
Content-Security-Policy	The Content-Security-Policy response header helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks.
X-Frame-Options	The X-Frame-Options response header can be used to indicate whether a browser should be allowed to render a page in a frame, iframe, embed or object.
Pragma	The Pragma header serves for backwards compatibility with the HTTP 1.0 caches that do not have a Cache-Control HTTP 1.1 header.
Expires	The Expires header contains the date and time after which the response is considered expired.
Keep-Alive	The Max field is the server configuration option that indicates the maximum number of requests allowed during a persistent connection. If set to 0, an unlimited amount of requests is allowed. The Timeout field is the server configuration option that describes the number of seconds the server waits for the next request from the same client on the same connection.

Connection	To accelerate transaction speed, you should use Keep-alive. For more information, refer to the "HTTP Keep Alive Settings" section.

Component 2: The XML Encoding and Version Details

The second component of the Response is the XML encoding and version details are described here:

<?xml version="1.0" encoding="UTF-8"?>

All text should comply with the standard XML character encoding rules as described in the XML specification at w3c.org Please refer to the section outlined below: https://www.w3.org/TR/xml/#charencoding

Component 3: The XML Response Body

The third component of the request is the **Response** root element, which includes the **Version** and XML **NameSpace** attributes and its sub-element as detailed below.

Version	<Response Version = "3"> is mandatory for each response as it defines the current version of the Datawire Secure Transport XML Interface. Any future versions of the Interface will have different versions for this attribute.
xmlns or prefixed with xmlns:	The XML Namespace attribute is as follows: xmlns=http://securetransport.dw/rcservice/xml

The Sub-Elements

The **Response** root element includes the **RespClientID**, **Status**, and **TransactionResponse** sub-elements as described in more detail below.

The RespClientID Sub-Element

The **RespClientID** sub-element is included in every response to identify and authenticate each response being sent through the Datawire Secure Transport Network back to the POS application.

Parameters

DID	The DID (Datawire Secure Transport ID) is a unique customer identifier generated by the Datawire Secure Transport and assigned to each client using the Datawire Secure Transport Network. This DID is obtained from the Datawire Secure Transport Vendor Integration Team (securetransport.integration@fiserv.com) or by the SRS process. The maximum DID length is 32 characters. The value of the DID element in the response echoes what appears in the request.
ClientRef	The ClientRef element identifies the packet sent by the application. This element needs to be unique to identify each transaction separately. A transaction counter is an acceptable method of creating this unique transaction identifier. For more information, contact securetransport.integration@fiserv.com. The value of the ClientRef element in the response echoes what appears in the request.

The Status Sub-Element

The second component of the **Response** element is the **Status** sub-element, which includes the **StatusCode**

attribute. The **StatusCode** attribute of the processed request is included in every Response as it describes the result of executed Request.

When a transaction is successfully passed from the application, through the Datawire Secure Transport Network to the Rapid Connect platform, back to the Datawire Secure Transport Network and parsed successfully by the Datawire Secure Transport Network, the Response's **StatusCode** reads "**OK**". This "OK" does not mean that the transaction was processed successfully at Rapid Connect or at the payment processor.

```
<Status StatusCode="OK"></Status>
```

When a transaction is unsuccessful (failed in the Datawire Secure Transport Network, or on the Rapid Connect platform, or at the payment processor), the response's **StatusCode** may be one of the following:

OK | AuthenticationError | UnknownServiceID | XMLError | OtherError | WrongSessionContext | Timeout

For more information, refer to the "Status Code Values" section.

The TransactionResponse Sub-Element

The third component of the "Response" root element is the **TransactionResponse** sub-element, which, when successful, contains the **ReturnCode** and **Payload** as described below and shown in the sample code later in the chapter.

Important: If the transaction is unsuccessful, there may not be a **TransactionResponse** sub-element. In this case, the **StatusCode** will contain the error details, which will help manage the error. For a sample of this error format, refer to the "Sample XML Error Response without a Return Code" section.

Parameters

ReturnCode	When the Return Code is not "000", an error has occurred. A sample ReturnCode is outlined in the "Sample XML Error Response With A Return Code" below. All the possible ReturnCodes are listed in the "Return Code Values" section.

Payload	The value of encoding attribute in a response echoes the encoding value of its corresponding request. The raw format of the Rapid Connect message is a standard xml message. The encoded format of the Rapid Connect xml message is carried in the Datawire Secure Transport Payload element. The encoding is specified by the Payload's encoding attribute, which could be one of two values: **xml_escape** or **cdata**. Any other encodings are not supported. Important: You must only apply the encoding once for the Payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly. **xml_escape:** This is the **default** encoding. <Payload Encoding="xml_escape">xml_escaped_RapidConnect_Response_Message</Payload> Note:xml_escaped_RapidConnect_Response_Message=xml_escaped(raw_RapidConnect_XML_Response_message) **cdata:** For **cdata** encoding, the intact raw Rapid Connect XML message is wrapped inside the <![**CDATA**[and]]>. <Payload Encoding="**cdata**"><![**CDATA**[raw_RapidConnect_XML_Response_message]]></Payload> **Important Encoding Details** For xml_escape encoding, any occurances of the character "<", ">" or "&" in the raw Rapid Connect XML message will be replaced by "<", ">" and "&" respectively. For cdata encoding: Be cautious: the raw Rapid Connect XML message CANNOT contain "]]>". Use xml_escape encoding if this sequence of characters appear in the raw Rapid Connect XML message. If the merchant application chooses cdata encoding, the Datawire Secure Transport Network will apply this encoding as well in its response – in this process Datawire Secure Transport Network will NOT check if the raw Rapid Connect XML response message contains "]]>". **Example** In the example below, you can see how the raw XML data is encoded differently: Raw XML message payload: "<RC>Hello, A & B</RC>" xml_escape encoded message payload: "<RC>Hello,A & B</RC>" **cdata** encoded message payload: "<![CDATA[<RC>Hello,A & B</RC>]]>"

The Initiate SessionResponse Sub-Element

The third component of the "Response" root element is the **InitiateSessionResponse** sub-element, which, when successful, contains the SessionContext, the ReturnCode, and **Payload** as described below and shown in the sample code later in the chapter. The Payload sub-element is optional.

Important: If the transaction is unsuccessful, there may not be an **InitiateSessionResponse** sub-element. In this case, the **StatusCode** will contain the error details that will help manage the error. For a sample of this error format, refer to the "Sample XML Error Response without a Return Code" section.

Parameters

SessionContext	The SessionContext should be used in the consequent SessionTransactions and TerminateSession. It guarantees that all the transactions in the session will go through the same route to the payment processor.
ReturnCode	When the Return Code is not "000", an error has occurred. A sample ReturnCode is outlined in the "Sample XML Error Response With A Return Code" below. All the possible ReturnCodes are listed in the "Return Code Values" section.
Payload	The value of encoding attribute in a response echoes the encoding value of its corresponding request. The raw format of the Rapid Connect message is a standard xml message. The encoded format of the Rapid Connect xml message is carried in the Datawire Secure Transport Payload element. The encoding is specified by the Payload's encoding attribute, which could be one of two values: **xml_escape** or **cdata**. Any other encodings are not supported. Important: You must only apply the encoding once for the Payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly. **xml_escape:** This is the **default** encoding. <Payload Encoding="xml_escape">xml_escaped_RapidConnect_Response_Message</Payload> Note:xml_escaped_RapidConnect_Response_Message=xml_escaped(raw_RapidConnect_XML_Response_message) **cdata:** For **cdata** encoding, the intact raw Rapid Connect XML message is wrapped inside the <![**CDATA**[and]]>. <Payload Encoding="**cdata**"><![**CDATA**[raw_RapidConnect_XML_Response_message]]></Payload> **Important Encoding Details** For xml_escape encoding, any occurances of the character "<", ">" or "&" in the raw Rapid Connect XML message will be replaced by "<", ">" and "&" respectively. For cdata encoding: Be cautious: the raw Rapid Connect XML message CANNOT contain "]]>". Use xml_escape encoding if this sequence of characters appear in the raw Rapid Connect XML message. If the merchant application chooses cdata encoding, the Datawire Secure Transport Network will apply this encoding as well in its response – in this process Datawire Secure Transport Network will NOT check if the raw Rapid Connect XML response message contains "]]>". **Example** In the example below, you can see how the raw XML data is encoded differently: Raw XML message payload: "<RC>Hello, A & B</RC>" xml_escape encoded message payload: "<RC>Hello,A & B</RC>" **cdata** encoded message payload: "<![CDATA[<RC>Hello,A & B</RC>]]>"

The SessionTransactionResponse Sub-Element

The third component of the "Response" root element is the **SessionTransactionResponse** sub-element, which, when successful, contains the **ReturnCode** and **Payload** as described below and shown in the sample code later in the chapter.

Important: If the transaction is unsuccessful, there may not be a **SessionTransactionResponse** sub-element. In this case, the **StatusCode** will contain the error details that will help manage the error. For a sample of this error format, refer to the "Sample XML Error Response without a Return Code" section.

Payload	The value of encoding attribute in a response echoes the encoding value of its corresponding request. The raw format of the *Rapid Connect* message is a standard xml message. The encoded format of the *Rapid Connect* xml message is carried in the Datawire Secure Transport Payload element. The encoding is specified by the Payload's encoding attribute, which could be one of two values: **xml_escape** or **cdata**. Any other encodings are not supported. Important: You must only apply the encoding once for the Payload value, which is the raw *Rapid Connect* XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly. **xml_escape:** This is the **default** encoding. <Payload Encoding="xml_escape">xml_escaped_RapidConnect_Response_Message</Payload> Note:xml_escaped_RapidConnect_Response_Message=xml_escaped(raw_RapidConnect_XML_Response_message) **cdata:** For **cdata** encoding, the intact raw Rapid Connect XML message is wrapped inside the <![**CDATA**[and]]>. <Payload Encoding="**cdata**"><![**CDATA**[raw_RapidConnect_XML_Response_message]]></Payload> **Important Encoding Details** For xml_escape encoding, any occurances of the character "<", ">" or "&" in the raw Rapid Connect XML message will be replaced by "<", ">" and "&" respectively. For cdata encoding: Be cautious: the raw Rapid Connect XML message CANNOT contain "]]>". Use xml_escape encoding if this sequence of characters appear in the raw Rapid Connect XML message. If the merchant application chooses cdata encoding, the Datawire Secure Transport Network will apply this encoding as well in its response – in this process Datawire Secure Transport Network will NOT check if the raw Rapid Connect XML response message contains "]]>". **Example** In the example below, you can see how the raw XML data is encoded differently: Raw XML message payload: "<RC>Hello, A & B</RC>" xml_escape encoded message payload: "<RC>Hello,A & B</RC> " **cdata** encoded message payload: "<![CDATA[<RC>Hello,A & B</RC>]]>"

The TerminateSessionResponse Sub-Element

The third component of the "Response" root element is the **TerminateSessionResponse** sub-element, which, when successful, contains the **ReturnCode** and **Payload** as described below and shown in the sample code later in the chapter. The Payload sub-element is optional.

Important: If the transaction is unsuccessful, there may not be a **TerminateSessionResponse** sub-element. In this case, the **StatusCode** will contain the error details that will help manage the error. For a sample of this error format, refer to the "Sample XML Error Response without a Return Code" section.

Payload	The value of encoding attribute in a response echoes the encoding value of its corresponding request. The raw format of the Rapid Connect message is a standard xml message. The encoded format of the Rapid Connect xml message is carried in the Datawire Secure Transport Payload element. The encoding is specified by the Payload's encoding attribute, which could be one of two values: **xml_escape** or **cdata**. Any other encodings are not supported. Important: You must only apply the encoding once for the Payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly. **xml_escape:** This is the **default** encoding. <Payload Encoding="xml_escape">xml_escaped_RapidConnect_Response_Message</Payload> Note:xml_escaped_RapidConnect_Response_Message=xml_escaped(raw_RapidConnect_XML_Response_message) **cdata:** For **cdata** encoding, the intact raw Rapid Connect XML message is wrapped inside the <![**CDATA** and]]>. <Payload Encoding="**cdata**"><![**CDATA**[raw_RapidConnect_XML_Response_message]]></Payload> **Important Encoding Details** For xml_escape encoding, any occurances of the character "<", ">" or "&" in the raw Rapid Connect XML message will be replaced by "<", ">" and "&" respectively. For cdata encoding: Be cautious: the raw Rapid Connect XML message CANNOT contain "]]>". Use xml_escape encoding if this sequence of characters appear in the raw Rapid Connect XML message. If the merchant application chooses cdata encoding, the Datawire Secure Transport Network will apply this encoding as well in its response – in this process Datawire Secure Transport Network will NOT check if the raw Rapid Connect XML response message contains "]]>". **Example** In the example below, you can see how the raw XML data is encoded differently: Raw XML message payload: "<RC>Hello, A & B</RC>" xml_escape encoded message payload: "<RC>Hello,A & B</RC> " **cdata** encoded message payload: "<![CDATA[<RC>Hello,A & B</RC>]]>"

Sample XML Response with XML_Escape Encoding

The sample below is an example of what a response with xml_escape encoding may look like.
Note: The sample below is only provided for your reference and thus contains only sample values.

The HTTP Response Headers included below are provided as samples and are subject to change based on industry requirements.

HTTP/1.1 200 OK

Date: Fri, 04 Mar 2022 21:12:57 GMT

Server: The Server returning the Response

Cache-Control: no-cache, no-store, must-revalidate

X-XSS-Protection: 1; mode=block

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000; includeSubdomains;

Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';

X-Frame-Options: SAMEORIGIN

Pragma: no-cache

Content-Length: length_of_body (actual length of HTTP message body)

Cache-Control: max-age=0

Expires: Fri, 04 Mar 2022 21:12:57 GMT

Keep-Alive: timeout=30, max=100

Connection: Keep-Alive

Content-Type: text/xml

```
<?xml version="1.0" encoding="UTF-8"?>
<Response Version="3" xmlns="http://securetransport.dw/rcservice/xml">
    <RespClientID>
        <DID>Sample_DID</DID>
        <ClientRef>Sample_ClientRef</ClientRef>
</RespClientID>
    <Status StatusCode="OK"></Status>
    <TransactionResponse>
        <ReturnCode>000</ReturnCode>
        <Payload Encoding="xml_escape">&lt;RC&gt;A &amp; B speaking&lt;/RC&gt;</Payload>
    </TransactionResponse>
</Response>
```

Note: The sample below of InitiateSessionResponse is only provided for your reference and thus contains only sample values.

HTTP/1.1 200 OK

Date: Fri, 04 Mar 2022 21:12:57 GMT

Server: The Server returning the Response

Cache-Control: no-cache, no-store, must-revalidate

X-XSS-Protection: 1; mode=block

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000; includeSubdomains;

Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';

X-Frame-Options: SAMEORIGIN

Pragma: no-cache

Content-Length: length_of_body (actual length of HTTP message body)

Cache-Control: max-age=0

Expires: Fri, 04 Mar 2022 21:12:57 GMT

Keep-Alive: timeout=30, max=100

Connection: Keep-Alive

Content-Type: text/xml


```
<?xml version="1.0" encoding="UTF-8"?>
<Response Version="3" xmlns="http://securetransport.dw/rcservice/xml">
    <RespClientID>
        <DID>Sample_DID</DID>
        <ClientRef>Sample_ClientRef</ClientRef>
    </RespClientID>
    <Status StatusCode="OK"></Status>
    <InitiateSessionResponse>
        <SessionContext>105|19388|15731|233</SessionContext>
        <ReturnCode>000</ReturnCode>
        <Payload Encoding="xml_escape">&lt;RC&gt;A &amp; B speaking&lt;/RC&gt;</Payload>
    </InitiateSessionResponse>
</Response>
```


Note: The sample below of SessionTransactionResponse is only provided for your reference and thus contains only
sample values.

HTTP/1.1 200 OK

Date: Fri, 04 Mar 2022 21:12:57 GMT

Server: The Server returning the Response

Cache-Control: no-cache, no-store, must-revalidate

X-XSS-Protection: 1; mode=block

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000; includeSubdomains;

Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';

X-Frame-Options: SAMEORIGIN

Pragma: no-cache

Content-Length: length_of_body (actual length of HTTP message body)

Cache-Control: max-age=0

Expires: Fri, 04 Mar 2022 21:12:57 GMT

Keep-Alive: timeout=30, max=100

Connection: Keep-Alive

Content-Type: text/xml

```
<?xml version="1.0" encoding="UTF-8"?>
<Response Version="3" xmlns="http://securetransport.dw/rcservice/xml">
    <RespClientID>
        <DID>Sample_DID</DID>
        <ClientRef>Sample_ClientRef</ClientRef>
    </RespClientID>
    <Status StatusCode="OK"></Status>
    <SessionTransactionResponse>
        <ReturnCode>000</ReturnCode>
        <Payload Encoding="xml_escape">&lt;RC&gt;A &amp; B speaking&lt;/RC&gt;</Payload>
    </SessionTransactionResponse>
</Response>
```


Note: The sample below of TerminateSessionResponse is only provided for your reference and thus contains only sample values.

```
HTTP/1.1 200 OK
Date: Fri, 04 Mar 2022 21:12:57 GMT
Server: The Server returning the Response
Cache-Control: no-cache, no-store, must-revalidate
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
Strict-Transport-Security: max-age=31536000; includeSubdomains;
Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';
X-Frame-Options: SAMEORIGIN
Pragma: no-cache
Content-Length: length_of_body (actual length of HTTP message body)
Cache-Control: max-age=0
Expires: Fri, 04 Mar 2022 21:12:57 GMT
Keep-Alive: timeout=30, max=100
Connection: Keep-Alive
Content-Type: text/xml
```


```
<?xml version="1.0" encoding="UTF-8"?>
<Response Version="3" xmlns="http://securetransport.dw/rcservice/xml">
    <RespClientID>
        <DID>Sample_DID</DID>
        <ClientRef>Sample_ClientRef</ClientRef>
    </RespClientID>
    <Status StatusCode="OK"></Status>
    <TerminateSessionResponse>
```

```
        <ReturnCode>000</ReturnCode>
        <Payload Encoding="xml_escape">&lt;RC&gt;A &amp; B speaking&lt;/RC&gt;</Payload>
    </TerminateSessionResponse>
</Response>
```

Sample XML Response with Cdata Encoding

Note: The sample below is only provided for your reference and thus contains only sample values.

```
HTTP/1.1 200 OK
Date: Fri, 04 Mar 2022 21:12:57 GMT
Server: The Server returning the Response
Cache-Control: no-cache, no-store, must-revalidate
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
Strict-Transport-Security: max-age=31536000; includeSubdomains;
Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';
X-Frame-Options: SAMEORIGIN
Pragma: no-cache
Content-Length: length_of_body (actual length of HTTP message body)
Cache-Control: max-age=0
Expires: Fri, 04 Mar 2022 21:12:57 GMT
Keep-Alive: timeout=30, max=100
Connection: Keep-Alive
Content-Type: text/xml

<?xml version="1.0" encoding="UTF-8"?>
<Response Version="3" xmlns="http://securetransport.dw/rcservice/xml">
    <RespClientID>
        <DID>Sample_DID</DID>
        <ClientRef>Sample_ClientRef</ClientRef>
    </RespClientID>
    <Status StatusCode="OK"></Status>
    <TransactionResponse>
        <ReturnCode>000</ReturnCode>
        <Payload Encoding="cdata"><![CDATA[<RC>A & B speaking</RC>]]></Payload>
    </TransactionResponse>
</Response>
```

Note: The sample below of InitiateSessionResponse is only provided for your reference and thus contains only sample values.

HTTP/1.1 200 OK

Date: Fri, 04 Mar 2022 21:12:57 GMT

Server: The Server returning the Response

Cache-Control: no-cache, no-store, must-revalidate

X-XSS-Protection: 1; mode=block

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000; includeSubdomains;

Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';

X-Frame-Options: SAMEORIGIN

Pragma: no-cache

Content-Length: length_of_body (actual length of HTTP message body)

Cache-Control: max-age=0

Expires: Fri, 04 Mar 2022 21:12:57 GMT

Keep-Alive: timeout=30, max=100

Connection: Keep-Alive

Content-Type: text/xml

```
<?xml version="1.0" encoding="UTF-8"?>
<Response Version="3" xmlns="http://securetransport.dw/rcservice/xml">
    <RespClientID>
        <DID>Sample_DID</DID>
        <ClientRef>Sample_ClientRef</ClientRef>
    </RespClientID>
    <Status StatusCode="OK"></Status>
    <InitiateSessionResponse>
        <SessionContext>105|19388|15731|233</SessionContext>
        <ReturnCode>000</ReturnCode>
        <Payload Encoding="cdata"><![CDATA[<RC>A & B speaking</RC>]]></Payload>
    </InitiateSessionResponse>
</Response>
```

Note: The sample below of SessionTransactionResponse is only provided for your reference and thus contains only
sample values.

HTTP/1.1 200 OK

Date: Fri, 04 Mar 2022 21:12:57 GMT

Server: The Server returning the Response

Cache-Control: no-cache, no-store, must-revalidate

X-XSS-Protection: 1; mode=block

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000; includeSubdomains;

Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';

X-Frame-Options: SAMEORIGIN

Pragma: no-cache

Content-Length: length_of_body (actual length of HTTP message body)

Cache-Control: max-age=0

Expires: Fri, 04 Mar 2022 21:12:57 GMT

Keep-Alive: timeout=30, max=100

Connection: Keep-Alive

Content-Type: text/xml

```
<?xml version="1.0" encoding="UTF-8"?>
<Response Version="3" xmlns="http://securetransport.dw/rcservice/xml">
    <RespClientID>
        <DID>Sample_DID</DID>
        <ClientRef>Sample_ClientRef</ClientRef>
    </RespClientID>
    <Status StatusCode="OK"></Status>
    <SessionTransactionResponse>
        <ReturnCode>000</ReturnCode>
        <Payload Encoding="cdata"><![CDATA[<RC>A & B speaking</RC>]]></Payload>
    </SessionTransactionResponse>
</Response>
```

Note: The sample below of TerminateSessionResponse is only provided for your reference and thus contains only sample values.

HTTP/1.1 200 OK

Date: Fri, 04 Mar 2022 21:12:57 GMT

Server: The Server returning the Response

Cache-Control: no-cache, no-store, must-revalidate

X-XSS-Protection: 1; mode=block

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000; includeSubdomains;

Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';

X-Frame-Options: SAMEORIGIN

Pragma: no-cache

Content-Length: length_of_body (actual length of HTTP message body)

Cache-Control: max-age=0

Expires: Fri, 04 Mar 2022 21:12:57 GMT

Keep-Alive: timeout=30, max=100

```
Connection: Keep-Alive
Content-Type: text/xml

<?xml version="1.0" encoding="UTF-8"?>
<Response Version="3" xmlns="http://securetransport.dw/rcservice/xml">
    <RespClientID>
        <DID>Sample_DID</DID>
        <ClientRef>Sample_ClientRef</ClientRef>
    </RespClientID>
    <Status StatusCode="OK"></Status>
    <TerminateSessionResponse>
        <ReturnCode>000</ReturnCode>
        <Payload Encoding="cdata"><![CDATA[<RC>A & B speaking</RC>]]></Payload>
    </TerminateSessionResponse>
</Response>
```

Error Response Components

A response containing an error is made up of the three major components listed below. For more information regarding the XML components, refer to the XML schema file. In addition, for a sample, refer to the samples at the end of this section.

Component 1: HTTP Response Header

The Secure Transport HTTP response headers are driven by industry standards and PCI regulatory requirements.

Datawire Secure Transport Network may add new HTTP response header(s), delete some HTTP response headers, or change values of some of the HTTP response headers. This may be required to comply with security and/or industry best practices. Any API user should bear this in mind and make its application conform to respective HTTP RFCs to allow robustness without affecting its customers.

The HTTP Response Headers included below are provided as samples and are subject to change based on industry requirements.

Parameters

Date	The day, month, year, 00:00:00 GMT of the Response.
Server	The Server returning the Response.
Content-Length	Length of HTTP message body.
Content-Type	The Content-Type must be text/xml or the Datawire Secure Transport Network will not accept the message.
Cache-Control	The Cache-Control header directives specify behavior intended to prevent caches from adversely interfering with the request or response. These directives typically override the default caching algorithms.

X-XSS-Protection	The X-XSS-Protection response header helps prevent reflected cross-site scripting attacks.
X-Content-Type-Options	The X-Content-Type-Options response header is a marker used by the server to indicate that the MIME types advertised in the Content-Type headers should not be changed and be followed.
Strict-Transport-Security	The Strict-Transport-Security response header allows for a more effective implementation of TLS by ensuring all communication takes place over a secure transport layer on the client side.
Content-Security-Policy	The Content-Security-Policy response header helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks.
X-Frame-Options	The X-Frame-Options response header can be used to indicate whether a browser should be allowed to render a page in a frame, iframe, embed or object.
Pragma	The Pragma header serves for backwards compatibility with the HTTP 1.0 caches that do not have a Cache-Control HTTP 1.1 header.
Expires	The Expires header contains the date and time after which the response is considered expired.
Keep-Alive	The Max field is the server configuration option that indicates the maximum number of requests allowed during a persistent connection. If set to 0, an unlimited amount of requests is allowed. The Timeout field is the server configuration option that describes the number of seconds the server waits for the next request from the same client on the same connection.
Connection	To accelerate transaction speed, you should use Keep-alive. For more information, refer to the "HTTP Keep Alive Settings" section.

Component 2: The XML Encoding and Version Details

The second component of an Error "Response" root element is the XML encoding and version details are described here: **<?xml version="1.0" encoding="UTF-8"?>**

All text should comply with the standard XML character encoding rules as described in the XML specification at w3c.org. Please refer to the section outlined below:

https://www.w3.org/TR/xml/#charencoding

Component 3: The XML Response Body

The third component is the **Response** root element, which includes the Version and XML NameSpace attributes, and its sub- elements as detailed below.

Version	<Response Version = "3"> is mandatory for each response as it defines the current version of the Datawire Secure Transport XML Interface. Any future versions of the Interface will have different versions for this attribute.
xmlns or prefixed with xmlns:	The XML Namespace attribute is as follows: xmlns=http://securetransport.dw/rcservice/xml

The Sub-Elements

The **Response** root includes the **RespClientID**, **Status**, and **TransactionResponse** sub-elements as described in more detail below.

The RespClientID Sub-Element

The **RespClientID** sub-element is included in every response to identify and authenticate each response being sent through the Datawire Secure Transport Network back to the POS application.

Parameters

DID	The DID (Datawire Secure Transport ID) is a unique customer identifier generated by Datawire Secure Transport and assigned to each application using the Datawire Secure Transport Network. This DID is obtained from the Datawire Secure Transport Vendor Integration Team (securetransport.integration@fiserv.com) or by the SRS process. The maximum DID length is 32 characters. The value of the DID element in the response echoes what appears in the request.
ClientRef	The ClientRef element identifies the packet sent by the application. This element needs to be unique to identify each transaction separately. A transaction counter is an acceptable method of creating this unique transaction identifier. For more information, contact securetransport.integration@fiserv.com. The value of the ClientRef element in the response echoes what appears in the request.

The Status Sub-Element

The second component of the "Response" root element is the **Status** sub-element, which includes the **StatusCode** attribute. The **StatusCode** attribute indicates the status of the processed request and is included in every response. The **Status** element describes the result of executed request and contains a readable description of the result code.

When a transaction is unsuccessful, the response's **StatusCode** may be one of the following:

OK | AuthenticationError | UnknownServiceID | XMLError | OtherError | WrongSessionContext | Timeout

For more information, refer to the "Status Code Values" section.

The TransactionResponse Sub-Element

The third component of the "Response" root element is the **TransactionResponse** sub-element, which, when successful, contains the **ReturnCode** and **Payload** as described below and shown in the sample code later in the chapter.

Important: If the transaction is unsuccessful, there may not be a **TransactionResponse** sub-element. In this case, the **StatusCode** will contain the error details that will help manage the error. For a sample of this error format, refer to the "Sample XML Error Response without a Return Code" section.

ReturnCode	When the Return Code is not "000", an error has occurred. All the available ReturnCodes are listed in the "Return Code Values" section.
Payload	If there is an error, there will be no "payload".

The InitiateSessionResponse Sub-Element

The third component of the "Response" root element is the **InitiateSessionResponse** sub-element, which, when successful, contains the SessionContext, the ReturnCode, and **Payload** as described below and shown in the sample code later in the chapter. The Payload sub-element is optional.

Important: If the transaction is unsuccessful, there may not be an **InitiateSessionResponse** sub-element. In this

case, the **StatusCode** will contain the error details that will help manage the error. For a sample of this error format, refer to the "Sample XML Error Response without a Return Code" section.

SessionContext	If there is an error, the SessionContext should be empty
ReturnCode	When the Return Code is not "000", an error has occurred. All the available ReturnCodes are listed in the "Return Code Values" section.
Payload	If there is an error, there will be no Payload element

The SessionTransactionResponse Sub-Element

The third component of the "Response" root element is the **SessionTransactionResponse** sub-element, which, when successful, contains the **ReturnCode** and **Payload** as described below and shown in the sample code later in the chapter.

Important: If the transaction is unsuccessful, there may not be a **SessionTransactionResponse** sub-element. In this case, the **StatusCode** will contain the error details that will help manage the error. For a sample of this error format, refer to the "Sample XML Error Response without a Return Code" section.

ReturnCode	When the Return Code is not "000", an error has occurred. All the available ReturnCodes are listed in the "Return Code Values" section.
Payload	If there is an error, there will be no "payload".

The TerminateSessionResponse Sub-Element

The third component of the "Response" root element is the **TerminateSessionResponse** sub-element, which, when successful, contains the **ReturnCode** and **Payload** as described below and shown in the sample code later in the chapter. The Payload sub-element is optional.

Important: If the transaction is unsuccessful, there may not be a **TerminateSessionResponse** sub-element. In this case, the **StatusCode** will contain the error details that will help manage the error. For a sample of this error format, refer to the "Sample XML Error Response without a Return Code" section.

ReturnCode	When the Return Code is not "000", an error has occurred. All the available ReturnCodes are listed in the "Return Code Values" section.
Payload	If there is an error, there will be no Payload

Sample XML Error Response with a Return Code

If an error is encountered, the XML response will contain a **ReturnCode** sub-element within the **TransactionResponse**, **InitiateSessionResponse**, **SessionTransactionResponse**, or **TerminateSessionResponse** element. Samples of these structures are included below.

XML_Escape Encoding Sample

Warning: The sample code is for your reference. This sample is not suitable for direct production use.

```
HTTP/1.1 200 OK

Date: Fri, 04 Mar 2022 21:12:57 GMT

Server: The Server returning the Response
```

Cache-Control: no-cache, no-store, must-revalidate

X-XSS-Protection: 1; mode=block

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000; includeSubdomains;

Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';

X-Frame-Options: SAMEORIGIN

Pragma: no-cache

Content-Length: length_of_body (actual length of HTTP message body)

Cache-Control: max-age=0

Expires: Fri, 04 Mar 2022 21:12:57 GMT

Keep-Alive: timeout=30, max=100

Connection: Keep-Alive

Content-Type: text/xml

```
<?xml version="1.0" encoding="UTF-8"?>
<Response Version="3" xmlns="http://securetransport.dw/rcservice/xml">
    <RespClientID>
        <DID>Sample_DID</DID>
        <ClientRef>Sample_ClientRef</ClientRef>
    </RespClientID>
    <Status StatusCode="OK"></Status>
    <TransactionResponse>
        <ReturnCode>202</ReturnCode>
        <Payload Encoding="xml_escape"></Payload>
    </TransactionResponse>
</Response>
```

Warning: The sample code of InitiateSessionResponse is for your reference. This sample is not suitable for direct production use.

HTTP/1.1 200 OK

Date: Fri, 04 Mar 2022 21:12:57 GMT

Server: The Server returning the Response

Cache-Control: no-cache, no-store, must-revalidate

X-XSS-Protection: 1; mode=block

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000; includeSubdomains;

Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';

X-Frame-Options: SAMEORIGIN

Pragma: no-cache

Content-Length: length_of_body (actual length of HTTP message body)

Cache-Control: max-age=0

Expires: Fri, 04 Mar 2022 21:12:57 GMT

Keep-Alive: timeout=30, max=100

Connection: Keep-Alive

Content-Type: text/xml

```
<?xml version="1.0" encoding="UTF-8"?>
<Response Version="3" xmlns="http://securetransport.dw/rcservice/xml">
    <RespClientID>
        <DID>Sample_DID</DID>
        <ClientRef>Sample_ClientRef</ClientRef>
    </RespClientID>
    <Status StatusCode="OK"></Status>
    <InitiateSessionResponse>
        <SessionContext></SessionContext>
        <ReturnCode>202</ReturnCode>
    </InitiateSessionResponse>
</Response>
```

Warning: The sample code of SessionTransactionResponse is for your reference. This sample is not suitable for direct production use.

HTTP/1.1 200 OK

Date: Fri, 04 Mar 2022 21:12:57 GMT

Server: The Server returning the Response

Cache-Control: no-cache, no-store, must-revalidate

X-XSS-Protection: 1; mode=block

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000; includeSubdomains;

Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';

X-Frame-Options: SAMEORIGIN

Pragma: no-cache

Content-Length: length_of_body (actual length of HTTP message body)

Cache-Control: max-age=0

Expires: Fri, 04 Mar 2022 21:12:57 GMT

Keep-Alive: timeout=30, max=100

Connection: Keep-Alive

Content-Type: text/xml

```
<?xml version="1.0" encoding="UTF-8"?>
<Response Version="3" xmlns="http://securetransport.dw/rcservice/xml">
    <RespClientID>
        <DID>Sample_DID</DID>
        <ClientRef>Sample_ClientRef</ClientRef>
    </RespClientID>
    <Status StatusCode="OK"></Status>
    <SessionTransactionResponse>
        <ReturnCode>006</ReturnCode>
        <Payload Encoding="xml_escape"></Payload>
    </SessionTransactionResponse>
</Response>
```


Warning: The sample code of TerminateSessionResponse is for your reference. This sample is not suitable for direct production use.

```
HTTP/1.1 200 OK
Date: Fri, 04 Mar 2022 21:12:57 GMT
Server: The Server returning the Response
Cache-Control: no-cache, no-store, must-revalidate
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
Strict-Transport-Security: max-age=31536000; includeSubdomains;
Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';
X-Frame-Options: SAMEORIGIN
Pragma: no-cache
Content-Length: length_of_body (actual length of HTTP message body)
Cache-Control: max-age=0
Expires: Fri, 04 Mar 2022 21:12:57 GMT
Keep-Alive: timeout=30, max=100
Connection: Keep-Alive
Content-Type: text/xml
```


```
<?xml version="1.0" encoding="UTF-8"?>
<Response Version="3" xmlns="http://securetransport.dw/rcservice/xml">
    <RespClientID>
        <DID>Sample_DID</DID>
        <ClientRef>Sample_ClientRef</ClientRef>
    </RespClientID>
    <Status StatusCode="OK"></Status>
    <TerminateSessionResponse>
```

```
        <ReturnCode>006</ReturnCode>
    </TerminateSessionResponse>
</Response>
```

Cdata Encoding Sample

Note: The sample below is only provided for your reference and thus contains only sample values.

```
HTTP/1.1 200 OK
Date: Fri, 04 Mar 2022 21:12:57 GMT
Server: The Server returning the Response
Cache-Control: no-cache, no-store, must-revalidate
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
Strict-Transport-Security: max-age=31536000; includeSubdomains;
Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';
X-Frame-Options: SAMEORIGIN
Pragma: no-cache
Content-Length: length_of_body (actual length of HTTP message body)
Cache-Control: max-age=0
Expires: Fri, 04 Mar 2022 21:12:57 GMT
Keep-Alive: timeout=30, max=100
Connection: Keep-Alive
Content-Type: text/xml

<?xml version="1.0" encoding="UTF-8"?>
<Response Version="3" xmlns="http://securetransport.dw/rcservice/xml">
    <RespClientID>
        <DID>Sample_DID</DID>
        <ClientRef>Sample_ClientRef</ClientRef>
    </RespClientID>
    <Status StatusCode="OK"></Status>
    <TransactionResponse>
        <ReturnCode>202</ReturnCode>
        <Payload Encoding="cdata"></Payload>
    </TransactionResponse>
</Response>
```

Note: The sample below of InitiateSessionResponse is only provided for your reference and thus contains only sample values.

```
HTTP/1.1 200 OK
```

Date: Fri, 04 Mar 2022 21:12:57 GMT

Server: The Server returning the Response

Cache-Control: no-cache, no-store, must-revalidate

X-XSS-Protection: 1; mode=block

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000; includeSubdomains;

Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';

X-Frame-Options: SAMEORIGIN

Pragma: no-cache

Content-Length: length_of_body (actual length of HTTP message body)

Cache-Control: max-age=0

Expires: Fri, 04 Mar 2022 21:12:57 GMT

Keep-Alive: timeout=30, max=100

Connection: Keep-Alive

Content-Type: text/xml

```
<?xml version="1.0" encoding="UTF-8"?>
<Response Version="3" xmlns="http://securetransport.dw/rcservice/xml">
    <RespClientID>
        <DID>Sample_DID</DID>
        <ClientRef>Sample_ClientRef</ClientRef>
    </RespClientID>
    <Status StatusCode="OK"></Status>
    <InitiateSessionResponse>
        <SessionContext></SessionContext>
        <ReturnCode>202</ReturnCode>
    </InitiateSessionResponse>
</Response>
```

Note: The sample below of SessionTransactionResponse is only provided for your reference and thus contains only
sample values.

HTTP/1.1 200 OK

Date: Fri, 04 Mar 2022 21:12:57 GMT

Server: The Server returning the Response

Cache-Control: no-cache, no-store, must-revalidate

X-XSS-Protection: 1; mode=block

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000; includeSubdomains;

Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';

X-Frame-Options: SAMEORIGIN

Pragma: no-cache

Content-Length: length_of_body (actual length of HTTP message body)

Cache-Control: max-age=0

Expires: Fri, 04 Mar 2022 21:12:57 GMT

Keep-Alive: timeout=30, max=100

Connection: Keep-Alive

Content-Type: text/xml

```
<?xml version="1.0" encoding="UTF-8"?>
<Response Version="3" xmlns="http://securetransport.dw/rcservice/xml">
    <RespClientID>
        <DID>Sample_DID</DID>
        <ClientRef>Sample_ClientRef</ClientRef>
    </RespClientID>
    <Status StatusCode="OK"></Status>
    <SessionTransactionResponse>
        <ReturnCode>006</ReturnCode>
        <Payload Encoding="cdata"></Payload>
    </SessionTransactionResponse>
</Response>
```

Note: The sample below of TerminateSessionResponse is only provided for your reference and thus contains only sample values.

HTTP/1.1 200 OK

Date: Fri, 04 Mar 2022 21:12:57 GMT

Server: The Server returning the Response

Cache-Control: no-cache, no-store, must-revalidate

X-XSS-Protection: 1; mode=block

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000; includeSubdomains;

Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';

X-Frame-Options: SAMEORIGIN

Pragma: no-cache

Content-Length: length_of_body (actual length of HTTP message body)

Cache-Control: max-age=0

Expires: Fri, 04 Mar 2022 21:12:57 GMT

Keep-Alive: timeout=30, max=100

```
Connection: Keep-Alive
Content-Type: text/xml

<?xml version="1.0" encoding="UTF-8"?>
<Response Version="3" xmlns="http://securetransport.dw/rcservice/xml">
    <RespClientID>
        <DID>Sample_DID</DID>
        <ClientRef>Sample_ClientRef</ClientRef>
    </RespClientID>
    <Status StatusCode="OK"></Status>
    <TerminateSessionResponse>
        <ReturnCode>006</ReturnCode>
    </TerminateSessionResponse>
</Response>
```

Sample XML Error Response without a Return Code

Note: The sample below is only provided for your reference and thus contains only sample values.

```
HTTP/1.1 200 OK
Date: Fri, 04 Mar 2022 21:12:57 GMT
Server: The Server returning the Response
Cache-Control: no-cache, no-store, must-revalidate
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
Strict-Transport-Security: max-age=31536000; includeSubdomains;
Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';
X-Frame-Options: SAMEORIGIN
Pragma: no-cache
Content-Length: length_of_body (actual length of HTTP message body)
Cache-Control: max-age=0
Expires: Fri, 04 Mar 2022 21:12:57 GMT
Keep-Alive: timeout=30, max=100
Connection: Keep-Alive
Content-Type: text/xml

<?xml version="1.0" encoding="UTF-8"?>
<Response Version="3" xmlns="http://securetransport.dw/rcservice/xml">
    <RespClientID>
        <DID>Sample_DID</DID>
```

```
        <ClientRef>Sample_ClientRef</ClientRef>
    </RespClientID>
    <Status StatusCode="XMLError">Request missing data: &lt;DID&gt;</Status>
</Response>
```

The Soap Transaction

In This Chapter

- What is a Datawire Secure Transport SOAP Transaction?
- The Datawire Secure Transport SOAP Protocol
- Sending SOAP Requests and Receiving SOAP Responses
- The SOAP Request Components
- Sample SOAP Request with XML-Encoding
- Sample SOAP Request with CData Encoding
- The SOAP Response Components
- Sample SOAP Response with XML-Encoding
- Sample SOAP Response with CData Encoding
- The SOAP Error Response Components
- Sample SOAP Error Response with Return Code
- Sample SOAP Error Response without Return Code

What Is a Datawire Secure Transport Soap Transaction?

A Datawire Secure Transport SOAP transaction contains all the transaction data necessary to process a transaction at the Rapid Connect platform. However, as it passes through the Datawire Secure Transport Network, this transaction data is never stored on the Network for future use. The transaction data, if needed, must be stored on the merchant's system so that it can be used for transactions such as reversals and settlements.

The Datawire Secure Transport Soap Protocol

A transaction using the Datawire Secure Transport SOAP Protocol is composed of a request-response pair. The Request and Response messages are sent using the W3C SOAP Version 1.1.

Each SOAP Request message is sent as an HTTP POST. The transaction is sent to the Datawire Secure Transport Network Servers (Edge Nodes) that accept the HTTPS (HTTP over SSL) request. The Datawire Secure Transport Network Servers (Edge Nodes) will then route the transaction through the Datawire Secure Transport Network to the Rapid Connect platform, and receive and route the appropriate Response back to the POS application.

A Simple Transaction is composed of a single request-response pair. Payment transactions that have a "simple" nature should be wrapped up and sent as Simple Transactions. A typical example of a Simple Transaction is a credit card or debit card authorization, reversal, completion, etc...

A Session Transaction is composed of multiple request-response pairs that must be delivered to the payment processing server in sequential order. Session transactions should only be used when a simple request/response type of interaction cannot fulfill the end-to-end processing case, e.g. there are many level 3 line items need to be carried over along with
a transaction.

To Send and Receive Simple Transactions:

1. The POS application wraps the **Transaction** Request in a SOAP document.
2. The POS application posts the SOAP document over HTTPS to the Datawire Secure Transport Network.

   Once the **Transaction** Request is received, the Datawire Secure Transport Network will route the Request to

the Rapid Connect platform (the one specified by the **ServiceID**).

3. The Rapid Connect platform returns the Response in its own format. The Datawire Secure Transport Network translates this Response into the appropriate SOAP document.

   If the transaction is successful, the **StatusCode** is "OK" and the **ReturnCode** is "000". The results of the transaction are returned to the POS application in the **TransactionResponse** sub-element of the **Response** element.

   However, if the transaction is unsuccessful, the ReturnCode and StatusCode can vary and you need to be prepared to manage the error. For more information, refer to the "SOAP Error Response Components" section. For sample error messages, refer to the "Sample SOAP Error Response with XML Escape Encoding" section and the "Sample SOAP Error Response with CData Encoding" section.

4. The POS application then closes the HTTPS connection if there are no additional transactions to be sent.

To Send and Receive Session Transactions:

1. The POS application wraps an **InitiateSession** Request in a SOAP document.
2. The POS application posts the SOAP document over HTTPS to the Datawire Secure Transport Network. Once the **InitiateSession** Request is received, the Datawire Secure Transport Network will establish a connection to and route the payload to the Rapid Connect platform (the one specified by the **ServiceID**).
3. The Rapid Connect platform returns the Response in its own format. The Datawire Secure Transport Network translates this Response into the appropriate SOAP document. If the transaction is successful, the **ReturnCode** is "000" and the results of the transaction are returned to the POS application in the **InitiateSessionResponse** sub-element of the **Response** element. The value of **SessionContext** element (a sub- element of InitiateSessionResponse) should be saved by POS application and will be used as is in subsequent session transactions. However, if the transaction is unsuccessful, the **ReturnCode** and **StatusCode** can vary and you need to be prepared to manage the error. For more information, refer to the "Return Code Values" section. As well as the "Status Code Values" section.
4. The POS application wraps a **SessionTransaction** Request with the **SessionContext** it received in the **InitiateSessionResponse** in an SOAP document.
5. The POS application posts the SOAP document over HTTPS to the Datawire Secure Transport Network. Once the **SessionTransaction** Request is received, the Datawire Secure Transport Network will route the payload to the Rapid Connect platform (through the same route of the **InitiateSession** transaction).
6. The Rapid Connect platform returns the Response in its own format. The Datawire Secure Transport Network translates this Response into the appropriate SOAP document. If the transaction is successful, the **ReturnCode** is "000" and the results of the transaction are returned to the POS application in the **SessionTransactionResponse** sub-element of the **Response** element. However, if the transaction is unsuccessful, the **ReturnCode** and **StatusCode** can vary and you need to be prepared to manage the error.
7. Repeat step 4 to 6 until the POS application is ready to end this session.
8. The POS application wraps a **TerminateSession** Request with the **SessionContext** it received in the **InitiateSessionResponse** in a SOAP document.
9. The POS application posts the SOAP document over HTTPS to the Datawire Secure Transport Network. Once the **TerminateSession** Request is received, the Datawire Secure Transport Network will route the payload to the Rapid Connect platform (through the same route of the **InitiateSession** transaction).
10. The Rapid Connect platform returns the Response in its own format. The Datawire Secure Transport Network translates this Response into the appropriate SOAP document. If the transaction is successful, the **ReturnCode** is "000" and the results of the transaction are returned to the POS application in the **TerminateSessionResponse** sub-element of the **Response** element. However, if the transaction is

unsuccessful, the **ReturnCode** and **StatusCode** can vary and you need to be prepared to manage the error.

11. Datawire Secure Transport Network closes the connection to the Rapid Connect platform.

The Soap Request Components

For more information regarding the SOAP components, refer to the WSDL file. In addition, for a sample, refer to the "Sample SOAP Request with XML Escape Encoding" section.

Component 1: The HTTP Request Header

The first component of the SOAP Request is the HTTP Request Header, which includes the minimum required fields described below.

Parameters

User-Agent	The User-Agent element identifies the application that has generated and is sending the transaction. Contact the Datawire Secure Transport Vendor Integration Team (securetransport.integration@fiserv.com) for more information.
Host	The Host parameter is the name of the server to which the request is sent, and it is derived from the URL. Specifically: if the request is sent to the URL https://abc.123.net /path/to/dwxml, then the contents of this field must be "abc.123.net". This parameter will be provided by securetransport.integration@fiserv.com and is required in order for the Request to be successful.
Connection	To optimize transaction speed, you may use Keep-alive. For more information, refer to the "HTTP Keep Alive Settings" section.
Cache-Control	Cache must be turned off because each time you send a new request; the application needs to receive a fresh response from our Datawire Secure Transport Network Servers (Edge Nodes). Applications cannot receive cached responses.
Content-Type	The Content-Type must be text/xml or the Datawire Secure Transport Network will not accept the message.
Content-Length	Specifies the exact size of the request body, in bytes.
SOAPAction	http://securetransport.dw/rcservice

Component 2: The XML Encoding and Version Details

The second component of the Request contains the XML encoding and version details as sampled here:

```
<?xml version="1.0" encoding="UTF-8"?>
```

All text should comply with the standard XML character encoding rules as described in the XML specification at w3c.org. Please refer to the section outlined here: https://www.w3.org/TR/xml/#charencoding

Component 3: The Soap Request Body

The **SOAP Envelope** element is the root element of the SOAP message and includes:

- The namespace: http://schemas.xmlsoap.org/soap/envelope/.
- The mandatory **Body** sub-element as described in more detail below.

The Body Sub-Element

The **Body** sub-element, which includes the Request sub-element, as detailed below.

The Request Sub- Element

The **Request** sub-element is made up of the **Version**, **ClientTimeout** and **NameSpace** attributes as well as the **ReqClientID**and **Transaction** sub-elements described in detail below.

Version	The Version attribute is the <Request Version = "3"> is mandatory for each request as it defines the current version of the Datawire Secure Transport Interface. Any future versions of the Interface will have different versions for this attribute.
ClientTimeout	The ClientTimeout attribute holds the number of seconds before the client will timeout due to inactivity.
xmlns or prefixed with xmlns:	The Namespace attribute is as follows: xmlns=http://securetransport.dw/rcservice/soap

The Reqclientid Sub-Element

The **ReqClientID** sub-element is included in every request to identify and authenticate each request being sent to the Datawire Secure Transport Network.

Parameters

DID	The DID (Datawire Secure Transport ID) is a unique customer identifier generated by Datawire Secure Transport and assigned to each application using the Datawire Secure Transport Network. This DID is obtained from the Datawire Secure Transport Vendor Integration Team (securetransport.integration@fiserv.com) or by the SRS process. The maximum DID length is 32 characters.
App	The App element identifies the application that has generated and is sending the transaction. This is a 15 char alphanumeric code that identifies each application and is provided by the Datawire Secure Transport Vendor Integration Team (securetransport.integration@fiserv.com).
Auth	The Auth element specifies the authentication information needed to process the transaction: AuthKey1 and AuthKey2. The Auth element format is AuthKey1 followed by "|" followed by AuthKey2 (AuthKey1|AuthKey2). RapidConnect Support maintains the authority of the ultimate explanation on how to populate AuthKey1 and AuthKey2 from RapidConnect system parameters, below is just a brief illustration about the content of AuthKey1 and AuthKey2: -- AuthKey1 is a field of up to 32 alpha-numeric characters. -- AuthKey2 is a fixed length zero padded (to the left) field of 8 alpha-numeric characters. -- AuthKey1 is composed of Group ID followed by Merchant ID. Group ID and Merchant ID are parameters assigned by the Processor/RapidConnect system, please contact RapidConnect Support for specific Group ID and Merchant ID requirements. -- AuthKey2 is composed of Terminal ID (Front End TID). Terminal ID is a parameter assigned by the Processor/RapidConnect system, please contact RapidConnect Support for theTerminal ID. -- As an example, when given RapidConnect system parameters: Group ID=10001, Merchant ID=1054217,

	Terminal ID=1032339, AuthKey1 should be populated as “100011054217”, AuthKey2 should be populated as “01032339”, the final Auth is “100011054217|01032339”.
ClientRef	The ClientRef element identifies the packet sent by the application. This element needs to be unique to identify each transaction separately. A transaction counter is an acceptable method of creating this unique transaction identifier. For more information, contact securetransport.integration@fiserv.com.

The Transaction Sub-Element

Transaction sub-element contains the **ServiceID** and Payload sub-elements as shown below.

Parameters

ServiceID	The ServiceID sub-element is an identifier that is used to identify the payment processor to which the transactions will be sent for all subsequent transactions. The ServiceID is provided by the Datawire Secure Transport VIP Team (securetransport.integration@fiserv.com).
Payload	Note: If the encoding attribute is not specified, the default encoding is **xml_escape**. The raw format of the Rapid Connect message is a standard xml message. The encoded format of the Rapid Connect xml message is carried in the Datawire Secure Transport Payload element. The encoding is specified by the Payload’s encoding attribute, which could be one of two values: **xml_escape** or **cdata**. Any other encodings are not supported. **xml_escape:** <Payload Encoding="xml_escape">xml_escaped_RapidConnect_Message</Payload> **where** xml_escaped_RapidConnect_Message = xml_escape(raw_RapidConnect_XML_message) **cdata:** For **cdata** encoding, the intact raw Rapid Connect XML message is wrapped inside the <![**CDATA**[and]]>. <Payload Encoding="**cdata**"><![**CDATA**[raw_RapidConnect_XML_message]]></Payload> **Important Encoding Details** You must only apply the encoding once for the Payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly. The value of encoding attribute in a response echoes the encoding value of its corresponding request. For xml_escape encoding, any occurances of the character "<", ">" or "&" in the raw Rapid Connect XML message will be replaced by "<", ">" and "&" respectively. For cdata encoding: Be cautious: the raw Rapid Connect XML message CANNOT contain "]]>". Use xml_escape encoding if this sequence of characters appear in the raw Rapid Connect XML message. If the merchant application chooses cdata encoding, the Datawire Secure Transport Network will apply this encoding as well in its response – in this process Datawire Secure Transport Network will NOT check if the raw Rapid Connect XML response message contains "]]>". **Examples** In the example below, you can see how the raw XML data is encoded differently: Raw XML message payload: "<RC>Hello, A & B</RC>" **xml_escape** encoded message payload: "<RC>Hello,A & B</RC> " **cdata** encoded message payload: "<![CDATA[<RC>Hello,A & B</RC>]]>".

The InitiateSession Sub-Element

For an InitiateSession request, the second sub-element of the "Request" root element is the **InitiateSession** sub-element which contains the **ServiceID** and the **Payload** sub-elements. The Payload sub-element is optional.

Parameters

ServiceID	The ServiceID sub-element is an identifier that is used to identify the payment processor to which the transactions will be sent for all subsequent transactions. The ServiceID is provided by the Datawire Secure Transport VIP Team (securetransport.integration@fiserv.com).
Payload	Note: If the encoding attribute is not specified, the default encoding is **xml_escape**. The raw format of the Rapid Connect message is a standard xml message. The encoded format of the Rapid Connect xml message is carried in the Datawire Secure Transport Payload element. The encoding is specified by the Payload's encoding attribute, which could be one of two values: **xml_escape** or **cdata**. Any other encodings are not supported. **xml_escape:** <Payload Encoding="xml_escape">xml_escaped_RapidConnect_Message</Payload> where xml_escaped_RapidConnect_Message = xml_escape(raw_RapidConnect_XML_message) **cdata:** For **cdata** encoding, the intact raw Rapid Connect XML message is wrapped inside the <![**CDATA** and]]>. <Payload Encoding="**cdata**"><![**CDATA**[raw_RapidConnect_XML_message]]></Payload> **Important Encoding Details** You must only apply the encoding once for the Payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly. The value of encoding attribute in a response echoes the encoding value of its corresponding request. For xml_escape encoding, any occurances of the character "<", ">" or "&" in the raw Rapid Connect XML message will be replaced by "<", ">" and "&" respectively. For cdata encoding: Be cautious: the raw Rapid Connect XML message CANNOT contain "]]>". Use xml_escape encoding if this sequence of characters appear in the raw Rapid Connect XML message. If the merchant application chooses cdata encoding, the Datawire Secure Transport Network will apply this encoding as well in its response – in this process Datawire Secure Transport Network will NOT check if the raw Rapid Connect XML response message contains "]]>". **Examples** In the example below, you can see how the raw XML data is encoded differently: Raw XML message payload "<RC>Hello, A & B</RC>" **xml_escape** encoded message payload: "<RC>Hello,A & B</RC>" **cdata** encoded message payload: "<![CDATA[<RC>Hello,A & B</RC>]]>".

The SessionTransaction Sub-Element

For a SessionTransaction request, the second sub-element of the “Request” root element is the **SessionTransaction** sub- element, which contains the **SessionContext** and the **Payload** sub-elements.

Parameters

SessionContext	The SessionContext is what has been returned in the response of the previous InitiateSession request. It guarantees that all the transactions in the session will go through the same route to the payment processor.
Payload	Note: If the encoding attribute is not specified, the default encoding is **xml_escape**. The raw format of the Rapid Connect message is a standard xml message. The encoded format of the Rapid Connect xml message is carried in the Datawire Secure Transport Payload element. The encoding is specified by the Payload’s encoding attribute, which could be one of two values: **xml_escape** or **cdata**. Any other encodings are not supported. **xml_escape:** <Payload Encoding="xml_escape">xml_escaped_RapidConnect_Message</Payload> **where** xml_escaped_RapidConnect_Message = xml_escape(raw_RapidConnect_XML_message) **cdata:** For **cdata** encoding, the intact raw Rapid Connect XML message is wrapped inside the <![**CDATA**[and]]>. <Payload Encoding="**cdata**"><![**CDATA**[raw_RapidConnect_XML_message]]></Payload> **Important Encoding Details** You must only apply the encoding once for the Payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly. The value of encoding attribute in a response echoes the encoding value of its corresponding request. For xml_escape encoding, any occurances of the character "<", “>” or "&" in the raw Rapid Connect XML message will be replaced by "<", ">" and "&" respectively. For cdata encoding: Be cautious: the raw Rapid Connect XML message CANNOT contain “]]>”. Use xml_escape encoding if this sequence of characters appear in the raw Rapid Connect XML message. If the merchant application chooses cdata encoding, the Datawire Secure Transport Network will apply this encoding as well in its response – in this process Datawire Secure Transport Network will NOT check if the raw Rapid Connect XML response message contains “]]>”. **Examples** In the example below, you can see how the raw XML data is encoded differently: Raw XML message payload: “<RC>Hello, A & B</RC>” **xml_escape** encoded message payload: “<RC>Hello,A & B</RC>” **cdata** encoded message payload: "<![CDATA[<RC>Hello,A & B</RC>]]>".

The TerminateSession Sub-Element

For a TerminateSession request, the second sub-element of the “Request” root element is the **TerminateSession** sub-element, which contains the SessionContext, and the **Payload** sub-elements. The Payload sub-element is optional.

Parameters

SessionContext	The SessionContext is what has been returned in the response of the previous InitiateSession request. It guarantees that all the transactions in the session will go through the same route to the payment processor.
Payload	Note: If the encoding attribute is not specified, the default encoding is **xml_escape**. The raw format of the Rapid Connect message is a standard xml message. The encoded format of the Rapid Connect xml message is carried in the Datawire Secure Transport Payload element. The encoding is specified by the Payload’s encoding attribute, which could be one of two values: **xml_escape** or **cdata**. Any other encodings are not supported. **xml_escape:** <Payload Encoding="xml_escape">xml_escaped_RapidConnect_Message</Payload> where xml_escaped_RapidConnect_Message = xml_escape(raw_RapidConnect_XML_message) **cdata:** For **cdata** encoding, the intact raw Rapid Connect XML message is wrapped inside the <![**CDATA**[and]]>. <Payload Encoding="**cdata**"><![**CDATA**[raw_RapidConnect_XML_message]]></Payload> **Important Encoding Details** You must only apply the encoding once for the Payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly. The value of encoding attribute in a response echoes the encoding value of its corresponding request. For xml_escape encoding, any occurances of the character "<", ">" or "&" in the raw Rapid Connect XML message will be replaced by "<", ">" and "&" respectively. For cdata encoding: Be cautious: the raw Rapid Connect XML message CANNOT contain "]]>". Use xml_escape encoding if this sequence of characters appear in the raw Rapid Connect XML message. If the merchant application chooses cdata encoding, the Datawire Secure Transport Network will apply this encoding as well in its response – in this process Datawire Secure Transport Network will NOT check if the raw Rapid Connect XML response message contains "]]>". **Examples** In the example below, you can see how the raw XML data is encoded differently: Raw XML message payload: "<RC>Hello, A & B</RC>" **xml_escape** encoded message payload: “<RC>Hello,A & B</RC>” **cdata** encoded message payload: "<![CDATA[<RC>Hello,A & B</RC>]]>".

Sample Soap Request with XML_Escape Encoding

The samples below is an example of what a Request may look like.
Note: The sample below is only provided for your reference and thus contains only sample values.

```
POST /xyz HTTP/1.1
User-Agent: AppName_VersionNumber
Host: Host Portion of URL
Connection: Keep-Alive
Cache-Control: no-cache
Content-Type: text/xml
Content-Length: length_of_body (actual length of HTTP message body)
SOAPAction: http://securetransport.dw/rcservice

<?xml version="1.0" encoding="UTF-8"?>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" >
<S:Body>
    <Request Version="3" ClientTimeout="Sample_TimeoutValue" xmlns="http://securetransport.dw/rcservice/soap">
        <ReqClientID>
                <DID>Sample_DID</DID>
                <App>Sample_AppName</App>
                <Auth>Sample_AuthKey1|Sample_AuthKey2</Auth>
                <ClientRef>Sample_ClientRef</ClientRef>
        </ReqClientID>
        <Transaction>
                <ServiceID>Sample_ServiceID</ServiceID>
                <Payload Encoding="xml_escape">&lt;RC&gt;Hello, A &amp; B&lt;/RC&gt;</Payload>
        </Transaction>
    </Request>
</S:Body>
</S:Envelope>
```

Note: The sample below of InitiateSession is only provided for your reference and thus contains only sample values.

```
POST /xyz HTTP/1.1
User-Agent: AppName_VersionNumber
Host: Host Portion of URL
Connection: Keep-Alive
Cache-Control: no-cache
Content-Type: text/xml
Content-Length: length_of_body (actual length of HTTP message body)
```

```
SOAPAction: http://securetransport.dw/rcservice

<?xml version="1.0" encoding="UTF-8"?>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" >
<S:Body>
    <Request Version="3" ClientTimeout="Sample_TimeoutValue"
    xmlns="http://securetransport.dw/rcservice/soap">
        <ReqClientID>
                <DID>Sample_DID</DID>
                <App>Sample_AppName</App>
                <Auth>Sample_AuthKey1|Sample_AuthKey2</Auth>
                >Sample_ClientRef</ClientRef>
        </ReqClientID>
        <InitiateSession>
                <ServiceID>Sample_ServiceID</ServiceID>
                <Payload Encoding="xml_escape">&lt;RC&gt;Hello, A &amp; B&lt;/RC&gt;</Payload>
        </InitiateSession>
    </Request>
</S:Body>
</S:Envelope>
```

Note: The sample below of SessionTransaction is only provided for your reference and thus contains only sample values.

```
POST /xyz HTTP/1.1
User-Agent: AppName_VersionNumber
Host: Host Portion of URL
Connection: Keep-Alive
Cache-Control: no-cache
Content-Type: text/xml
Content-Length: length_of_body (actual length of HTTP message body)
SOAPAction: http://securetransport.dw/rcservice

<?xml version="1.0" encoding="UTF-8"?>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" >
<S:Body>
    <Request Version="3" ClientTimeout="Sample_TimeoutValue" xmlns="http://securetransport.dw/rcservice/soap">
        <ReqClientID>
                <DID>Sample_DID</DID>
                <App>Sample_AppName</App>
                <Auth>Sample_AuthKey1|Sample_AuthKey2</Auth>
```

```
            <ClientRef>Sample_ClientRef</ClientRef>
        </ReqClientID>
        <SessionTransaction>
            <SessionContext>105|19388|15731|233</SessionContext>
            <Payload Encoding="xml_escape">&lt;RC&gt;Hello, A &amp; B&lt;/RC&gt;</Payload>
        </SessionTransaction>
    </Request>
</S:Body>
</S:Envelope>
```

Note: The sample below of TerminateSession is only provided for your reference and thus contains only sample values.

```
POST /xyz HTTP/1.1
User-Agent: AppName_VersionNumber
Host: Host Portion of URL
Connection: Keep-Alive
Cache-Control: no-cache
Content-Type: text/xml
Content-Length: length_of_body (actual length of HTTP message body)
SOAPAction: http://securetransport.dw/rcservice

<?xml version="1.0" encoding="UTF-8"?>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" >
<S:Body>
    <Request Version="3" ClientTimeout="Sample_TimeoutValue" xmlns="http://securetransport.dw/rcservice/soap">
        <ReqClientID>
            <DID>Sample_DID</DID>
            <App>Sample_AppName</App>
            <Auth>Sample_AuthKey1|Sample_AuthKey2</Auth>
            <ClientRef>Sample_ClientRef</ClientRef>
        </ReqClientID>
        <TerminateSession>
            <SessionContext>105|19388|15731|233</SessionContext>
            <Payload Encoding="xml_escape">&lt;RC&gt;Hello, A &amp; B&lt;/RC&gt;</Payload>
        </TerminateSession>
    </Request>
</S:Body>
</S:Envelope>
```

Sample Soap Request with Cdata Encoding

Note: The sample below is only provided for your reference and thus contains only sample values.

```
POST /xyz HTTP/1.1
User-Agent: AppName_VersionNumber
Host: Host Portion of URL
Connection: Keep-Alive
Cache-Control: no-cache
Content-Type: text/xml
Content-Length: length_of_body
SOAPAction: http://securetransport.dw/rcservice

<?xml version="1.0" encoding="UTF-8?>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
<S:Body>
    <Request ClientTimeout="Sample_TimeoutValue" Version="3" xmlns="http://securetransport.dw/rcservice/soap">
        <ReqClientID>
                <DID>Sample_DID</DID>
                <App>Sample_AppName</App>
                <Auth>Sample_Authkey1|Sample_Authkey2</Auth>
                <ClientRef>Sample_ClientRef</ClientRef>
        </ReqClientID>
        <Transaction>
                <ServiceID>Sample_ServiceID</ServiceID>
                <Payload Encoding="cdata"><![CDATA[<RC>Hello, A & B</RC>]]></Payload>
        </Transaction>
    </Request>
</S:Body>
</S:Envelope>
```

Note: The sample below of InitiateSession is only provided for your reference and thus contains only sample values.

```
POST /xyz HTTP/1.1
User-Agent: AppName_VersionNumber
Host: Host Portion of URL
Connection: Keep-Alive
Cache-Control: no-cache
Content-Type: text/xml
Content-Length: length_of_body
```

```
SOAPAction: http://securetransport.dw/rcservice

<?xml version="1.0" encoding="UTF-8?>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
<S:Body>
    <Request ClientTimeout="Sample_TimeoutValue" Version="3" xmlns="http://securetransport.dw/rcservice/soap">
        <ReqClientID>
                <DID>Sample_DID</DID>
                <App>Sample_AppName</App>
                <Auth>Sample_AuthKey1|Sample_AuthKey2</Auth>
                <ClientRef>Sample_ClientRef</ClientRef>
        </ReqClientID>
        <InitiateSession>
                <ServiceID>Sample_ServiceID</ServiceID>
                <Payload Encoding="cdata"><![CDATA[<RC>Hello, A & B</RC>]]></Payload>
        </InitiateSession>
    </Request>
</S:Body>
</S:Envelpe>
```

Note: The sample below of SessionTransaction is only provided for your reference and thus contains only sample values.

```
POST /xyz HTTP/1.1
User-Agent: AppName_VersionNumber
Host: Host Portion of URL
Connection: Keep-Alive
Cache-Control: no-cache
Content-Type: text/xml
Content-Length: length_of_body
SOAPAction: http://securetransport.dw/rcservice

<?xml version="1.0" encoding="UTF-8?>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
<S:Body>
    <Request ClientTimeout="Sample_TimeoutValue" Version="3" xmlns="http://securetransport.dw/rcservice/soap">
        <ReqClientID>
                <DID>Sample_DID</DID>
                <App>Sample_AppName</App>
                <Auth>Sample_AuthKey1|Sample_AuthKey2</Auth>
                <ClientRef>Sample_ClientRef</ClientRef>
```

```
        </ReqClientID>
        <SessionTransaction>
                <SessionContext>105|19388|15731|233</SessionContext>
                <Payload Encoding="cdata"><![CDATA[<RC>Hello, A & B</RC>]]></Payload>
        </SessionTransaction>
    </Request>
</S:Body>
</S:Envelope>
```

Note: The sample below of TerminateSession is only provided for your reference and thus contains only sample values.

```
POST /xyz HTTP/1.1
User-Agent: AppName_VersionNumber
Host: Host Portion of URL
Connection: Keep-Alive
Cache-Control: no-cache
Content-Type: text/xml
Content-Length: length_of_body
SOAPAction: http://securetransport.dw/rcservice

<?xml version="1.0" encoding="UTF-8?>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
<S:Body>
    <Request ClientTimeout="Sample_TimeoutValue" Version="3" xmlns="http://securetransport.dw/rcservice/soap">
        <ReqClientID>
                <DID>Sample_DID</DID>
                <App>Sample_AppName</App>
                <Auth>Sample_AuthKey1|Sample_AuthKey2</Auth>
                <ClientRef>Sample_ClientRef</ClientRef>
        </ReqClientID>
        <TerminateSession>
                <SessionContext>105|19388|15731|233</SessionContext>
                <Payload Encoding="cdata"><![CDATA[<RC>Hello, A & B</RC>]]></Payload>
        </TerminateSession>
    </Request>
</S:Body>
</S:Envelope>
```

The Soap Response Components

A successful response contains the three major components listed below. For more information regarding the SOAP components, refer to the WSDL file. In addition, for a sample, refer to "The SOAP Response Components" section.

Component 1: The HTTP Response Header

The Secure Transport HTTP response headers are driven by industry standards and PCI regulatory requirements.

Datawire Secure Transport Network may add new HTTP response header(s), delete some HTTP response headers, or change values of some of the HTTP response headers. This may be required to comply with security and/or industry best practices. Any API user should bear this in mind and make its application conform to respective HTTP RFCs to allow robustness without affecting its customers.

The HTTP Response Headers included below are provided as samples and are subject to change based on industry requirements.

Parameters

Date	The day, month, year, 00:00:00 GMT of the Response.
Server	The Server returning the Response.
Content-Length	Specifies the exact size of the request body, in bytes.
Content-Type	The Content-Type must be text/xml.
Cache-Control	The Cache-Control header directives specify behavior intended to prevent caches from adversely interfering with the request or response. These directives typically override the default caching algorithms.
X-XSS-Protection	The X-XSS-Protection response header helps prevent reflected cross-site scripting attacks.
X-Content-Type-Options	The X-Content-Type-Options response header is a marker used by the server to indicate that the MIME types advertised in the Content-Type headers should not be changed and be followed.
Strict-Transport-Security	The Strict-Transport-Security response header allows for a more effective implementation of TLS by ensuring all communication takes place over a secure transport layer on the client side.
Content-Security-Policy	The Content-Security-Policy response header helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks.
X-Frame-Options	The X-Frame-Options response header can be used to indicate whether a browser should be allowed to render a page in a frame, iframe, embed or object.
Pragma	The Pragma header serves for backwards compatibility with the HTTP 1.0 caches that do not have a Cache-Control HTTP 1.1 header.
Expires	The Expires header contains the date and time after which the response is considered expired.
Keep-Alive	The Max field is the server configuration option that indicates the maximum number of requests allowed during a persistent connection. If set to 0, an unlimited amount of requests is allowed. The Timeout field is the server configuration option that describes the number of seconds the

	server waits for the next request from the same client on the same connection.
Connection	To accelerate transaction speed, you should use Keep-alive. For more information, refer to the ““HTTP Keep Alive Settings” section.

Component 2: The XML Encoding and Version Details

The second component of the Request contains the XML encoding and version details as sampled here:

```
<?xml version="1.0" encoding="UTF-8"?>
```

All text should comply with the standard XML character encoding rules as described in the XML specification at w3c.org. Please refer to the section outlined here: https://www.w3.org/TR/xml/#charencoding

Component 3: The Soap Response Body

The **SOAP Envelope** element is the root element of the SOAP message and includes:

- The namespace: http://schemas.xmlsoap.org/soap/envelope/.
- The mandatory **Body** sub-element as described in more detail below.

The Body Sub-Element

Body sub-element includes the **Version** and **NameSpace** attributes as described in the table below as well as the **Response** sub-elements detailed below.

Version	<Response Version = “3”> is mandatory for each response as it defines the current version of the Datawire Secure Transport XML Interface. Any future versions of the Interface will have different versions for this attribute.
xmlns or prefixed with xmlns:	The Namespace attribute is as follows: xmlns=http://securetransport.dw/rcservice/soap

The Response Sub-Element

The **Response** sub-element is made up of the **RespClientID, Status** and **TransactionResponse** sub-elements.

The RespClientID Sub-Element

The **RespClientID** sub-element is included in every response to identify and authenticate each response being sent through the Datawire Secure Transport Network back to the POS application.

Parameters

DID	The DID (Datawire Secure Transport ID) is a unique customer identifier generated by the Datawire Secure Transport and assigned to each client using the Datawire Secure Transport Network. This DID is obtained from the Datawire Secure Transport Vendor Integration Team (securetransport.integration@fiserv.com) or by the SRS process. The maximum DID length is 32 characters. The value of the DID element in the response echoes what appears in the request.
ClientRef	The ClientRef element identifies the packet sent by the application. This element needs to be unique to identify each transaction separately. A transaction counter is an acceptable method of creating this unique transaction identifier. For more information, contact securetransport.integration@fiserv.com. The value of the ClientRef element in the response echoes what appears in the request

The Status Sub-Element

The second component of the **Response** sub-element is the **Status** sub-element, which includes the **StatusCode** attribute. The **StatusCode** attribute indicates the status of the processed request and is included in every Response as it describes the result of executed Request.

When a transaction is successfully passed from the applicaction, through the Datawire Secure Transport Network to the payment processor, back to the Datawire Secure Transport Network and parsed successfully by the Datawire Secure Transport Network, the Response's **StatusCode** reads "**OK**". This "OK" does not mean that the transaction was processed successfully at Rapid Connect or at the payment processor.

```
<Status StatusCode="OK"></Status>
```

When a transaction is unsuccessful (failed in the Datawire Secure Transport Network, or on the Rapid Connect platform, or at the payment processor), the response's **StatusCode** may be one of the following:

OK | AuthenticationError | UnknownServiceID | XMLError | OtherError |
WrongSessionContext | Timeout

For more information, refer to the "Status Code Values" section.

The TransactionResponse Sub-Element

The third component of the **Response** sub-element is the **TransactionResponse** sub-element, which contains the **ReturnCode** and **Payload** described below and shown in the sample code later in the chapter.

Note: If the transaction is unsuccessful, there may not be a **TransactionResponse** sub-element. In this case, the **StatusCode** will contain the error details that will help manage the error.

Parameters

ReturnCode	The ReturnCode describes the error that has occurred as shown in the sample at the end of the chapter. All the possible ReturnCodes are listed in the "Return Code Values" section. Important: If the transaction is unsuccessful, there may not be a ReturnCode value. In this case, the StatusCode will contain the error details, which will help manage the error. For a sample of this error format, refer to the "Sample SOAP Error Response Without a Return Code" section.
Payload	The value of encoding attribute in a response echoes the encoding value of its corresponding request. The raw format of the Rapid Connect message is a standard xml message. The encoded format of the Rapid Connect xml message is carried in the Datawire Secure Transport Payload element. The encoding is specified by the Payload's encoding attribute, which could be one of two values: **xml_escape** or **cdata**. Any other encodings are not supported. Important: You must only apply the encoding once for the Payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly. **xml_escape:** This is the **default** encoding. <Payload Encoding="xml_escape">xml_escaped_RapidConnect_Response_Message</Payload> Note:xml_escaped_RapidConnect_Response_Message=xml_escaped(raw_RapidConnect_XML_Response_message) **cdata:** For **cdata** encoding, the intact raw Rapid Connect XML message is wrapped inside the <![**CDATA**[and]]>. <Payload Encoding="**cdata**"><![**CDATA**[raw_RapidConnect_XML_Response_message]]></Payload> **Important Encoding Details** For xml_escape encoding, any occurances of the character "<", ">" or "&" in the raw Rapid Connect XML message will be replaced by "<", ">" and "&" respectively. For cdata encoding: Be cautious: the raw Rapid Connect XML message CANNOT contain "]]>". Use xml_escape encoding if this sequence of characters appear in the raw Rapid Connect XML message. If the merchant application chooses cdata encoding, the Datawire Secure Transport Network will apply this encoding as well in its response – in this process Datawire Secure Transport Network will NOT check if the raw Rapid Connect XML response message contains "]]>". **Example** In the example below, you can see how the raw XML data is encoded differently: Raw XML message payload: "<RC>Hello, A & B</RC>" xml_escape encoded message payload: "<RC>Hello,A & B</RC> " **cdata** encoded message payload: "<![CDATA[<RC>Hello,A & B</RC>]]>".

The InitiateSessionResponse Sub-Element

The third component of the "Response" root element is the **InitiateSessionResponse** sub-element, which, when successful, contains the SessionContext, the ReturnCode, and **Payload** as described below and shown in the sample code later in the chapter. The Payload sub-element is optional.

Important: If the transaction is unsuccessful, there may not be an **InitiateSessionResponse** sub-element. In this case, the **StatusCode** will contain the error details that will help manage the error.

Parameters

SessionContext	The SessionContext should be used in the consequent SessionTransactions and TerminateSession. It guarantees that all the transactions in the session will go through the same route to the payment processor.
ReturnCode	The ReturnCode describes the error that has occurred as shown in the sample at the end of the chapter. All the possible ReturnCodes are listed in the "Return Code Values" section. Important: If the transaction is unsuccessful, there may not be a ReturnCode value. In this case, the StatusCode will contain the error details, which will help manage the error. For a sample of this error format, refer to the "Sample SOAP Error Response Without a Return Code" section.
Payload	The value of encoding attribute in a response echoes the encoding value of its corresponding request. The raw format of the Rapid Connect message is a standard xml message. The encoded format of the Rapid Connect xml message is carried in the Datawire Secure Transport Payload element. The encoding is specified by the Payload's encoding attribute, which could be one of two values: **xml_escape** or **cdata**. Any other encodings are not supported. Important: You must only apply the encoding once for the Payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly. **xml_escape:** This is the **default** encoding. <Payload Encoding="xml_escape">xml_escaped_RapidConnect_Response_Message</Payload> Note:xml_escaped_RapidConnect_Response_Message=xml_escaped(raw_RapidConnect_XML_Response_message) **cdata:** For **cdata** encoding, the intact raw Rapid Connect XML message is wrapped inside the <![**CDATA**[and]]>. <Payload Encoding="**cdata**"><![**CDATA**[raw_RapidConnect_XML_Response_message]]></Payload> Important Encoding Details For xml_escape encoding, any occurances of the character "<", ">" or "&" in the raw Rapid Connect XML message will be replaced by "<", ">" and "&" respectively. For cdata encoding: Be cautious: the raw Rapid Connect XML message CANNOT contain "]]>". Use xml_escape encoding if this sequence of characters appear in the raw Rapid Connect XML message. If the merchant application chooses cdata encoding, the Datawire Secure Transport Network will apply this encoding as well in its response – in this process Datawire Secure Transport Network will NOT check if the raw Rapid Connect XML response message contains "]]>". **Example** In the example below, you can see how the raw XML data is encoded differently: Raw XML message payload: "<RC>Hello, A & B</RC>" xml_escape encoded message payload: "<RC>Hello,A & B</RC> " **cdata** encoded message payload: "<![CDATA[<RC>Hello,A & B</RC>]]>".

The SessionTransactionResponse Sub-Element

The third component of the "Response" root element is the **SessionTransactionResponse** sub-element, which, when successful, contains the **ReturnCode** and **Payload** as described below and shown in the sample code later in the chapter.

Important: If the transaction is unsuccessful, there may not be a **SessionTransactionResponse** sub-element. In this

case, the **StatusCode** will contain the error details that will help manage the error.

Parameters

ReturnCode	The ReturnCode describes the error that has occurred as shown in the sample at the end of the chapter. All the possible ReturnCodes are listed in the "Return Code Values" section. Important: If the transaction is unsuccessful, there may not be a ReturnCode value. In this case, the StatusCode will contain the error details, which will help manage the error. For a sample of this error format, refer to the "Sample SOAP Error Response Without a Return Code" section.
Payload	The value of encoding attribute in a response echoes the encoding value of its corresponding request. The raw format of the Rapid Connect message is a standard xml message. The encoded format of the Rapid Connect xml message is carried in the Datawire Secure Transport Payload element. The encoding is specified by the Payload's encoding attribute, which could be one of two values: **xml_escape** or **cdata**. Any other encodings are not supported. Important: You must only apply the encoding once for the Payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly. **xml_escape:** This is the **default** encoding. <Payload Encoding="xml_escape">xml_escaped_RapidConnect_Response_Message</Payload> Note:xml_escaped_RapidConnect_Response_Message=xml_escaped(raw_RapidConnect_XML_Response_message) **cdata:** For **cdata** encoding, the intact raw Rapid Connect XML message is wrapped inside the <![**CDATA**[and]]>. <Payload Encoding="**cdata**"><![**CDATA**[raw_RapidConnect_XML_Response_message]]></Payload> **Important Encoding Details** For xml_escape encoding, any occurances of the character "<", ">" or "&" in the raw Rapid Connect XML message will be replaced by "<", ">" and "&" respectively. For cdata encoding: Be cautious: the raw Rapid Connect XML message CANNOT contain "]]>". Use xml_escape encoding if this sequence of characters appear in the raw Rapid Connect XML message. If the merchant application chooses cdata encoding, the Datawire Secure Transport Network will apply this encoding as well in its response – in this process Datawire Secure Transport Network will NOT check if the raw Rapid Connect XML response message contains "]]>". **Example** In the example below, you can see how the raw XML data is encoded differently: Raw XML message payload: "<RC>Hello, A & B</RC>" xml_escape encoded message payload: "<RC>Hello,A & B</RC> " **cdata** encoded message payload: "<![CDATA[<RC>Hello,A & B</RC>]]>".

The TerminateSessionResponse Sub-Element

The third component of the "Response" root element is the **TerminateSessionResponse** sub-element, which, when successful, contains the **ReturnCode** and **Payload** as described below and shown in the sample code later in the chapter. The Payload sub-element is optional.

Important: If the transaction is unsuccessful, there may not be a **TerminateSessionResponse** sub-element. In this

case, the **StatusCode** will contain the error details that will help manage the error.

Parameters

ReturnCode	The ReturnCode describes the error that has occurred as shown in the sample at the end of the chapter. All the possible ReturnCodes are listed in the "Return Code Values" section. Important: If the transaction is unsuccessful, there may not be a ReturnCode value. In this case, the StatusCode will contain the error details, which will help manage the error. For a sample of this error format, refer to the "Sample SOAP Error Response Without a Return Code" section.
Payload	The value of encoding attribute in a response echoes the encoding value of its corresponding request. The raw format of the Rapid Connect message is a standard xml message. The encoded format of the Rapid Connect xml message is carried in the Datawire Secure Transport Payload element. The encoding is specified by the Payload's encoding attribute, which could be one of two values: **xml_escape** or **cdata**. Any other encodings are not supported. Important: You must only apply the encoding once for the Payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly. **xml_escape:** This is the **default** encoding. <Payload Encoding="xml_escape">xml_escaped_RapidConnect_Response_Message</Payload> Note:xml_escaped_RapidConnect_Response_Message=xml_escaped(raw_RapidConnect_XML_Response_message) **cdata:** For **cdata** encoding, the intact raw Rapid Connect XML message is wrapped inside the <![**CDATA**[and]]>. <Payload Encoding="**cdata**"><![**CDATA**[raw_RapidConnect_XML_Response_message]]></Payload> Important Encoding Details For xml_escape encoding, any occurances of the character "<", ">" or "&" in the raw Rapid Connect XML message will be replaced by "<", ">" and "&" respectively. For cdata encoding: Be cautious: the raw Rapid Connect XML message CANNOT contain "]]>". Use xml_escape encoding if this sequence of characters appear in the raw Rapid Connect XML message. If the merchant application chooses cdata encoding, the Datawire Secure Transport Network will apply this encoding as well in its response – in this process Datawire Secure Transport Network will NOT check if the raw Rapid Connect XML response message contains "]]>". **Example** In the example below, you can see how the raw XML data is encoded differently: Raw XML message payload: "<RC>Hello, A & B</RC>" xml_escape encoded message payload: "<RC>Hello,A & B</RC> " **cdata** encoded message payload: "<![CDATA[<RC>Hello,A & B</RC>]]>".

Sample Response with XML_Escape Encoding

The samples below is an example of what the Responses may look like.
Note: The sample below is only provided for your reference and thus contains only sample values.

HTTP/1.1 200 OK

Date: Fri, 04 Mar 2022 21:12:57 GMT

Server: The Server returning the Response

Cache-Control: no-cache, no-store, must-revalidate

X-XSS-Protection: 1; mode=block

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000; includeSubdomains;

Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';

X-Frame-Options: SAMEORIGIN

Pragma: no-cache

Content-Length: length_of_body (actual length of HTTP message body)

Cache-Control: max-age=0

Expires: Fri, 04 Mar 2022 21:12:57 GMT

Keep-Alive: timeout=30, max=100

Connection: Keep-Alive

Content-Type: text/xml

```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
<S:Body>
    <Response Version="3" xmlns="http://securetransport.dw/rcservice/soap">
        <RespClientID>
                <DID>Sample_DID</DID>
                <ClientRef>Sample_ClientRef</ClientRef>
        </RespClientID>
        <Status StatusCode="OK"></Status>
        <TransactionResponse>
                <ReturnCode>000</ReturnCode>
                <Payload Encoding="xml_escape">&lt;RC&gt;A &amp; B speaking&lt;/RC&gt;</Payload>
        </TransactionResponse>
    </Response>
</S:Body>
</S:Envelope>
```

Note: The sample below of InitiateSessionResponse is only provided for your reference and thus contains only sample values.

HTTP/1.1 200 OK

Date: Fri, 04 Mar 2022 21:12:57 GMT

Server: The Server returning the Response

Cache-Control: no-cache, no-store, must-revalidate

X-XSS-Protection: 1; mode=block

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000; includeSubdomains;

Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';

X-Frame-Options: SAMEORIGIN

Pragma: no-cache

Content-Length: length_of_body (actual length of HTTP message body)

Cache-Control: max-age=0

Expires: Fri, 04 Mar 2022 21:12:57 GMT

Keep-Alive: timeout=30, max=100

Connection: Keep-Alive

Content-Type: text/xml

```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
<S:Body>
    <Response Version="3" xmlns="http://securetransport.dw/rcservice/soap">
        <RespClientID>
                <DID>Sample_DID</DID>
                <ClientRef>Sample_ClientRef</ClientRef>
        </RespClientID>
        <Status StatusCode="OK"></Status>
        <InitiateSessionResponse>
                <SessionContext>105|19388|15731|233</SessionContext>
                <ReturnCode>000</ReturnCode>
                <Payload Encoding="xml_escape">&lt;RC&gt;A &amp; B speaking&lt;/RC&gt;</Payload>
        </InitiateSessionResponse>
    </Response>
</S:Body>
</S:Envelope>s
```

Note: The sample below of SessionTransactionResponse is only provided for your reference and thus contains only
sample values.

HTTP/1.1 200 OK

Date: Fri, 04 Mar 2022 21:12:57 GMT

Server: The Server returning the Response

Cache-Control: no-cache, no-store, must-revalidate

X-XSS-Protection: 1; mode=block

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000; includeSubdomains;

Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';

X-Frame-Options: SAMEORIGIN

Pragma: no-cache

Content-Length: length_of_body (actual length of HTTP message body)

Cache-Control: max-age=0

Expires: Fri, 04 Mar 2022 21:12:57 GMT

Keep-Alive: timeout=30, max=100

Connection: Keep-Alive

Content-Type: text/xml

```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
<S:Body>
    <Response Version="3" xmlns="http://securetransport.dw/rcservice/soap">
        <RespClientID>
                <DID>Sample_DID</DID>
                <ClientRef>Sample_ClientRef</ClientRef>
        </RespClientID>
        <Status StatusCode="OK"></Status>
        <SessionTransactionResponse>
                <ReturnCode>000</ReturnCode>
                <Payload Encoding="xml_escape">&lt;RC&gt;A &amp; B speaking&lt;/RC&gt;</Payload>
        </SessionTransactionResponse>
    </Response>
</S:Body>
</S:Envelope>
```

Note: The sample below of TerminateSessionResponse is only provided for your reference and thus contains only sample values.

HTTP/1.1 200 OK

Date: Fri, 04 Mar 2022 21:12:57 GMT

Server: The Server returning the Response

Cache-Control: no-cache, no-store, must-revalidate

X-XSS-Protection: 1; mode=block

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000; includeSubdomains;

Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';

X-Frame-Options: SAMEORIGIN

Pragma: no-cache

Content-Length: length_of_body (actual length of HTTP message body)

Cache-Control: max-age=0

Expires: Fri, 04 Mar 2022 21:12:57 GMT

Keep-Alive: timeout=30, max=100

Connection: Keep-Alive

Content-Type: text/xml

```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
<S:Body>
    <Response Version="3" xmlns="http://securetransport.dw/rcservice/soap">
        <RespClientID>
                <DID>Sample_DID</DID>
                <ClientRef>Sample_ClientRef</ClientRef>
        </RespClientID>
        <Status StatusCode="OK"></Status>
        <TerminateSessionResponse>
                <ReturnCode>000</ReturnCode>
                <Payload Encoding="xml_escape">&lt;RC&gt;A &amp; B speaking&lt;/RC&gt;</Payload>
        </TerminateSessionResponse>
    </Response>
</S:Body></S:Envelope>
```

Sample Response with Cdata Encoding

Note: The sample below is only provided for your reference and thus contains only sample values.

HTTP/1.1 200 OK

Date: Fri, 04 Mar 2022 21:12:57 GMT

Server: The Server returning the Response

Cache-Control: no-cache, no-store, must-revalidate

X-XSS-Protection: 1; mode=block

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000; includeSubdomains;

Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';

X-Frame-Options: SAMEORIGIN

Pragma: no-cache

Content-Length: length_of_body (actual length of HTTP message body)

Cache-Control: max-age=0

Expires: Fri, 04 Mar 2022 21:12:57 GMT

Keep-Alive: timeout=30, max=100

Connection: Keep-Alive

Content-Type: text/xml


```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
<S:Body>
    <Response Version="3" xmlns="http://securetransport.dw/rcservice/soap">
        <RespClientID>
                <DID>Sample_DID</DID>
                <ClientRef>Sample_ClientRef</ClientRef>
        </RespClientID>
        <Status StatusCode="OK"></Status>
        <TransactionResponse>
                <ReturnCode>000</ReturnCode>
                <Payload Encoding="cdata"><![CDATA[<RC>A & B speaking</RC>]]></Payload>
        </TransactionResponse>
    </Response>
</S:Body></S:Envelope>
```


Note: The sample below of InitiateSessionResponse is only provided for your reference and thus contains only sample values.

HTTP/1.1 200 OK

Date: Fri, 04 Mar 2022 21:12:57 GMT

Server: The Server returning the Response

Cache-Control: no-cache, no-store, must-revalidate

X-XSS-Protection: 1; mode=block

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000; includeSubdomains;

Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';

X-Frame-Options: SAMEORIGIN

Pragma: no-cache

Content-Length: length_of_body (actual length of HTTP message body)

Cache-Control: max-age=0

Expires: Fri, 04 Mar 2022 21:12:57 GMT

Keep-Alive: timeout=30, max=100

Connection: Keep-Alive

Content-Type: text/xml

```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
<S:Body>
    <Response Version="3" xmlns="http://securetransport.dw/rcservice/soap">
        <RespClientID>
                <DID>Sample_DID</DID>
                <ClientRef>Sample_ClientRef</ClientRef>
        </RespClientID>
        <Status StatusCode="OK"></Status>
        <InitiateSessionResponse>
                <SessionContext>105|19388|15731|233</SessionContext>
                <ReturnCode>000</ReturnCode>
                <Payload Encoding="cdata"><![CDATA[<RC>A & B speaking</RC>]]></Payload>
        </InitiateSessionResponse>
    </Response>
</S:Body>
</S:Envelope>
```


Note: The sample below of SessionTransactionResponse is only provided for your reference and thus contains only
sample values.

```
HTTP/1.1 200 OK
Date: Fri, 04 Mar 2022 21:12:57 GMT
Server: The Server returning the Response
Cache-Control: no-cache, no-store, must-revalidate
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
Strict-Transport-Security: max-age=31536000; includeSubdomains;
Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';
X-Frame-Options: SAMEORIGIN
Pragma: no-cache
Content-Length: length_of_body (actual length of HTTP message body)
Cache-Control: max-age=0
Expires: Fri, 04 Mar 2022 21:12:57 GMT
Keep-Alive: timeout=30, max=100
Connection: Keep-Alive
Content-Type: text/xml
```


```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
<S:Body>
    <Response Version="3" xmlns="http://securetransport.dw/rcservice/soap">
```

```
        <RespClientID>
                <DID>Sample_DID</DID>
                <ClientRef>Sample_ClientRef</ClientRef>
        </RespClientID>
        <Status StatusCode="OK"></Status>
        <SessionTransactionResponse>
                <ReturnCode>000</ReturnCode>
                <Payload Encoding="cdata"><![CDATA[<RC>A & B speaking</RC>]]></Payload>
        </SessionTransactionResponse>
    </Response>
</S:Body>
</S:Envelope>
```

NOTE: The sample below of TerminateSessionResponse is only provided for your reference and thus contains only
sample values.

```
HTTP/1.1 200 OK
Date: Fri, 04 Mar 2022 21:12:57 GMT
Server: The Server returning the Response
Cache-Control: no-cache, no-store, must-revalidate
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
Strict-Transport-Security: max-age=31536000; includeSubdomains;
Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';
X-Frame-Options: SAMEORIGIN
Pragma: no-cache
Content-Length: length_of_body (actual length of HTTP message body)
Cache-Control: max-age=0
Expires: Fri, 04 Mar 2022 21:12:57 GMT
Keep-Alive: timeout=30, max=100
Connection: Keep-Alive
Content-Type: text/xml

<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
<S:Body>
    <Response Version="3" xmlns="http://securetransport.dw/rcservice/soap">
        <RespClientID>
                <DID>Sample_DID</DID>
                <ClientRef>Sample_ClientRef</ClientRef>
        </RespClientID>
```

```
        <Status StatusCode="OK"></Status>
        <TerminateSessionResponse>
                <ReturnCode>000</ReturnCode>
                <Payload Encoding="cdata"><![CDATA[<RC>A & B speaking</RC>]]></Payload>
        </TerminateSessionResponse>
    </Response>
</S:Body>
</S:Envelope>
```

SOAP Error Response Components

For more information regarding the SOAP components, refer to the WSDL file. In addition, for a sample, refer to the samples at the end of this section.

Component 1: The HTTP Response Header

The Secure Transport HTTP response headers are driven by industry standards and PCI regulatory requirements.

Datawire Secure Transport Network may add new HTTP response header(s), delete some HTTP response headers, or change values of some of the HTTP response headers. This may be required to comply with security and/or industry best practices. Any API user should bear this in mind and make its application conform to respective HTTP RFCs to allow robustness without affecting its customers.

The HTTP Response Headers included below are provided as samples and are subject to change based on industry requirements.

Parameters

Date	The day, month, year, 00:00:00 GMT of the Response.
Server	The Server returning the Response.
Content-Length	Specifies the exact size of the request body, in bytes.
Content-Type	The Content-Type must be text/xml.
Cache-Control	The Cache-Control header directives specify behavior intended to prevent caches from adversely interfering with the request or response. These directives typically override the default caching algorithms.
X-XSS-Protection	The X-XSS-Protection response header helps prevent reflected cross-site scripting attacks.
X-Content-Type-Options	The X-Content-Type-Options response header is a marker used by the server to indicate that the MIME types advertised in the Content-Type headers should not be changed and be followed.
Strict-Transport-Security	The Strict-Transport-Security response header allows for a more effective implementation of TLS by ensuring all communication takes place over a secure transport layer on the client side.
Content-Security-Policy	The Content-Security-Policy response header helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks.

X-Frame-Options	The X-Frame-Options response header can be used to indicate whether a browser should be allowed to render a page in a frame, iframe, embed or object.
Pragma	The Pragma header serves for backwards compatibility with the HTTP 1.0 caches that do not have a Cache-Control HTTP 1.1 header.
Expires	The Expires header contains the date and time after which the response is considered expired.
Keep-Alive	The Max field is the server configuration option that indicates the maximum number of requests allowed during a persistent connection. If set to 0, an unlimited amount of requests is allowed. The Timeout field is the server configuration option that describes the number of seconds the server waits for the next request from the same client on the same connection.
Connection	To accelerate transaction speed, you should use Keep-alive. For more information, refer to the ““HTTP Keep Alive Settings” section.

Component 2: The XML Encoding and Version Details

The second component of the Request contains the XML encoding and version details as sampled here:

```
<?xml version="1.0" encoding="UTF-8"?>
```

All text should comply with the standard XML character encoding rules as described in the XML specification at w3c.org. Please refer to the section outlined here: https://www.w3.org/TR/xml/#charencoding

Component 3: The SOAP Response Body

The **SOAP Envelope** element is the root element of the SOAP message and includes:

- The envelope: http://schemas.xmlsoap.org/soap/envelope/;
- The mandatory **Body** sub-element as described below.

The Body Sub-Element

The mandatory **Body** sub-element contains the **Version** and the **Response** sub-element as described below.

Version	<Response Version = “3”> is mandatory for each response as it defines the current version of the Datawire Secure Transport XML Interface. Any future versions of the Interface will have different versions for this attribute.
xmlns or prefixed with xmlns:	The Namespace attribute is as follows: xmlns=http://securetransport.dw/rcservice/soap

The Response Sub-Element

The Response sub-element is made up of the RespClientID, StatusCode, and TransactionResponse sub-elements.

The RespClientID Sub-Element

The first component of the **Response** sub-element is the **RespClientID**. The **RespClientID** element is included in every response to identify and authenticate each response being sent through the Datawire Secure Transport Network back to the POS application.

Parameters

DID	The DID (Datawire Secure Transport ID) is a unique customer identifier generated by the Datawire Secure Transport and assigned to each client using the Datawire Secure Transport Network. This DID is obtained from the Datawire Secure Transport Vendor Integration Team (securetransport.integration@fiserv.com) or by the SRS process. The maximum DID length is 32 characters. The value of the DID element in the response echoes what appears in the request.
ClientRef	The ClientRef element identifies the packet sent by the application. This element needs to be unique to identify each transaction separately. A transaction counter is an acceptable method of creating this unique transaction identifier. For more information, contact securetransport.integration@fiserv.com. The value of the ClientRef element in the response echoes what appears in the request.

The Status Sub-Element

The second component of the **Response** sub-element is the **Status** sub-element, which includes the **StatusCode** attribute. The **StatusCode** attribute indicates the status of the processed request and is included in every Response as it describes the result of executed Request.

When a transaction is unsuccessful, the response's **StatusCode** may be one of the following:

OK | AuthenticationError | UnknownServiceID | XMLError | OtherError | WrongSessionContext | Timeout

For more information, refer to the "Status Code Values" section.

The TransactionResponse Sub-Element

The third component of the **Response** sub-element is the **TransactionResponse** sub-element, which, when successful, contains the **ReturnCode** and **Payload** described below and shown in the sample code later in the chapter.

Note: If the transaction is unsuccessful, there may not be a **TransactionResponse** sub-element. In this case, the **StatusCode** will contain the error details that will help manage the error. For a sample of this error format, refer to the "Sample SOAP Error Response without a Return Code" section.

Parameters

ReturnCode	The ReturnCode describes the error that has occurred as shown in the sample at the end of the chapter. All the possible ReturnCodes are listed in the "Return Code Values" section. Important: If the transaction is unsuccessful, there may not be a ReturnCode value. In this case, the StatusCode will contain the error details that will help manage the error. For a sample of this error format, refer to the "Sample SOAP Error Response Without a Return Code" section.
Payload	If there is an error, there will be no "payload".

The InitiateSessionResponse Sub-Element

The third component of the "Response" root element is the **InitiateSessionResponse** sub-element, which, when successful, contains the SessionContext, the ReturnCode, and **Payload** as described below and shown in the sample code later in the chapter. The Payload sub-element is optional.

Important: If the transaction is unsuccessful, there may not be an **InitiateSessionResponse** sub-element. In this case, the **StatusCode** will contain the error details that will help manage the error. For a sample of this error format, refer to the "Sample SOAP Error Response without a Return Code" section.

Parameters

SessionContext	If there is an error, the SessionContext should be empty.
ReturnCode	The ReturnCode describes the error that has occurred as shown in the sample at the end of the chapter. All the possible ReturnCodes are listed in the "Return Code Values" section. Important: If the transaction is unsuccessful, there may not be a ReturnCode value. In this case, the StatusCode will contain the error details that will help manage the error. For a sample of this error format, refer to the "Sample SOAP Error Response Without a Return Code" section.
Payload	If there is an error, there will be no Payload element.

The SessionTransactionResponse Sub-Element

The third component of the "Response" root element is the **SessionTransactionResponse** sub-element, which, when successful, contains the **ReturnCode** and **Payload** as described below and shown in the sample code later in the chapter.

Important: If the transaction is unsuccessful, there may not be a **SessionTransactionResponse** sub-element. In this case, the **StatusCode** will contain the error details that will help manage the error. For a sample of this error format, refer to the "Sample SOAP Error Response without a Return Code" section.

Parameters

ReturnCode	The ReturnCode describes the error that has occurred as shown in the sample at the end of the chapter. All the possible ReturnCodes are listed in "Return Code Values" section. Important: If the transaction is unsuccessful, there may not be a ReturnCode value. In this case, the StatusCode will contain the error details, which will help manage the error. For a sample of this error format, refer to the "Sample SOAP Error Response Without a Return Code" section.
Payload	If there is an error, there will be no "payload".

The TerminateSessionResponse Sub-Element

The third component of the "Response" root element is the **TerminateSessionResponse** sub-element, which, when successful, contains the **ReturnCode** and **Payload** as described below and shown in the sample code later in the chapter. The Payload sub- element is optional.

Important: If the transaction is unsuccessful, there may not be a **TerminateSessionResponse** sub-element. In this case, the **StatusCode** will contain the error details that will help manage the error. For a sample of this error format, refer to the "Sample SOAP Error Response without a Return Code" section.

Parameters

ReturnCode	The ReturnCode describes the error that has occurred as shown in the sample at the end of the chapter. All the possible ReturnCodes are listed in "Return Code Values" section. **Important**: If the transaction is unsuccessful, there may not be a ReturnCode value. In this case, the StatusCode will contain the error details that will help manage the error. For a sample of this error format, refer to the "Sample SOAP Error Response Without a Return Code" section.
Payload	If there is an error, there will be no Payload element.

Sample SOAP Error Response with XML_ Escape Encoding

Note: The sample below is only provided for your reference and thus contains only sample values.

HTTP/1.1 200 OK

Date: Fri, 04 Mar 2022 21:12:57 GMT

Server: The Server returning the Response

Cache-Control: no-cache, no-store, must-revalidate

X-XSS-Protection: 1; mode=block

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000; includeSubdomains;

Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';

X-Frame-Options: SAMEORIGIN

Pragma: no-cache

Content-Length: length_of_body (actual length of HTTP message body)

Cache-Control: max-age=0

Expires: Fri, 04 Mar 2022 21:12:57 GMT

Keep-Alive: timeout=30, max=100

Connection: Keep-Alive

Content-Type: text/xml

```
<?xml version="1.0" encoding="UTF-8"?>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
<S:Body>
    <Response Version="3" xmlns="http://securetransport.dw/rcservice/soap">
        <RespClientID>
                <DID>Sample_DID</DID>
                <ClientRef>Sample_ClientRef</ClientRef>
        </RespClientID>
        <Status StatusCode="OK"></Status>
        <TransactionResponse>
                <ReturnCode>202</ReturnCode>
                <Payload Encoding="xml_escape"></Payload>
```

```
        </TransactionResponse>
    </Response>
</S:Body>
</S:Envelope>
```

Note: The sample below of InitiateSessionResponse is only provided for your reference and thus contains only sample values.

```
HTTP/1.1 200 OK
Date: Fri, 04 Mar 2022 21:12:57 GMT
Server: The Server returning the Response
Cache-Control: no-cache, no-store, must-revalidate
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
Strict-Transport-Security: max-age=31536000; includeSubdomains;
Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';
X-Frame-Options: SAMEORIGIN
Pragma: no-cache
Content-Length: length_of_body (actual length of HTTP message body)
Cache-Control: max-age=0
Expires: Fri, 04 Mar 2022 21:12:57 GMT
Keep-Alive: timeout=30, max=100
Connection: Keep-Alive
Content-Type: text/xml

<?xml version="1.0" encoding="UTF-8"?>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
<S:Body>
    <Response Version="3" xmlns="http://securetransport.dw/rcservice/soap">
        <RespClientID>
                <DID>Sample_DID</DID>
                <ClientRef>Sample_ClientRef</ClientRef>
        </RespClientID>
        <Status StatusCode="OK"></Status>
        <InitiateSessionResponse>
                <SessionContext></SessionContext>
                <ReturnCode>202</ReturnCode>
        </InitiateSessionResponse>
    </Response>
</S:Body>
</S:Envelope>
```

Note: The sample below of SessionTransactionResponse is only provided for your reference and thus contains only
sample values.

```
HTTP/1.1 200 OK
Date: Fri, 04 Mar 2022 21:12:57 GMT
Server: The Server returning the Response
Cache-Control: no-cache, no-store, must-revalidate
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
Strict-Transport-Security: max-age=31536000; includeSubdomains;
Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';
X-Frame-Options: SAMEORIGIN
Pragma: no-cache
Content-Length: length_of_body (actual length of HTTP message body)
Cache-Control: max-age=0
Expires: Fri, 04 Mar 2022 21:12:57 GMT
Keep-Alive: timeout=30, max=100
Connection: Keep-Alive
Content-Type: text/xml

<?xml version="1.0" encoding="UTF-8"?>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
<S:Body>
    <Response Version="3" xmlns="http://securetransport.dw/rcservice/soap">
        <RespClientID>
                <DID>Sample_DID</DID>
                <ClientRef>Sample_ClientRef</ClientRef>
        </RespClientID>
        <Status StatusCode="OK"></Status>
        <SessionTransactionResponse>
                <ReturnCode>006</ReturnCode>
                <Payload Encoding="xml_escape"></Payload>
        </SessionTransactionResponse>
    </Response>
</S:Body>
</S:Envelope>
```

Note: The sample below of TerminateSessionResponse is only provided for your reference and thus contains only sample values.

```
HTTP/1.1 200 OK
```

Date: Fri, 04 Mar 2022 21:12:57 GMT

Server: The Server returning the Response

Cache-Control: no-cache, no-store, must-revalidate

X-XSS-Protection: 1; mode=block

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000; includeSubdomains;

Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';

X-Frame-Options: SAMEORIGIN

Pragma: no-cache

Content-Length: length_of_body (actual length of HTTP message body)

Cache-Control: max-age=0

Expires: Fri, 04 Mar 2022 21:12:57 GMT

Keep-Alive: timeout=30, max=100

Connection: Keep-Alive

Content-Type: text/xml

```
<?xml version="1.0" encoding="UTF-8"?>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
<S:Body>
    <Response Version="3" xmlns="http://securetransport.dw/rcservice/soap">
        <RespClientID>
                <DID>Sample_DID</DID>
                <ClientRef>Sample_ClientRef</ClientRef>
        </RespClientID>
        <Status StatusCode="OK"></Status>
        <TerminateSessionResponse>
                <ReturnCode>006</ReturnCode>
        </TerminateSessionResponse>
    </Response>
</S:Body>
</S:Envelope>
```

Sample SOAP Error Response with Cdata Encoding

NOTE: The sample below is only provided for your reference and thus contains only sample values.

HTTP/1.1 200 OK

Date: Fri, 04 Mar 2022 21:12:57 GMT

Server: The Server returning the Response

Cache-Control: no-cache, no-store, must-revalidate

X-XSS-Protection: 1; mode=block

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000; includeSubdomains;

Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';

X-Frame-Options: SAMEORIGIN

Pragma: no-cache

Content-Length: length_of_body (actual length of HTTP message body)

Cache-Control: max-age=0

Expires: Fri, 04 Mar 2022 21:12:57 GMT

Keep-Alive: timeout=30, max=100

Connection: Keep-Alive

Content-Type: text/xml

```
<?xml version="1.0" encoding="UTF-8"?>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/"><S:Body>
    <Response Version="3" xmlns="http://securetransport.dw/rcservice/soap">
        <RespClientID>
                <DID>Sample_DID</DID>
                <ClientRef>Sample_ClientRef</ClientRef>
        </RespClientID>
        <Status StatusCode="OK"></Status>
        <TransactionResponse>
                <ReturnCode>202</ReturnCode>
                <Payload Encoding="cdata"></Payload>
        </TransactionResponse>
    </Response>
</S:Body>
</S:Envelope>
```

Note: The sample below of InitiateSessionResponse is only provided for your reference and thus contains only sample values.

HTTP/1.1 200 OK

Date: Fri, 04 Mar 2022 21:12:57 GMT

Server: The Server returning the Response

Cache-Control: no-cache, no-store, must-revalidate

X-XSS-Protection: 1; mode=block

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000; includeSubdomains;

Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';

X-Frame-Options: SAMEORIGIN

Pragma: no-cache

Content-Length: length_of_body (actual length of HTTP message body)

Cache-Control: max-age=0

Expires: Fri, 04 Mar 2022 21:12:57 GMT

Keep-Alive: timeout=30, max=100

Connection: Keep-Alive

Content-Type: text/xml


```
<?xml version="1.0" encoding="UTF-8"?>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/"><S:Body>
    <Response Version="3" xmlns="http://securetransport.dw/rcservice/soap">
        <RespClientID>
                <DID>Sample_DID</DID>
                <ClientRef>Sample_ClientRef</ClientRef>
        </RespClientID>
        <Status StatusCode="OK"></Status>
        <InitiateSessionResponse>
                <SessionContext></SessionContext>
                <ReturnCode>202</ReturnCode>
        </InitiateSessionResponse>
    </Response>
</S:Body>
</S:Envelope>
```


Note: The sample below of SessionTransactionResponse is only provided for your reference and thus contains only
sample values.

HTTP/1.1 200 OK

Date: Fri, 04 Mar 2022 21:12:57 GMT

Server: The Server returning the Response

Cache-Control: no-cache, no-store, must-revalidate

X-XSS-Protection: 1; mode=block

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000; includeSubdomains;

Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';

X-Frame-Options: SAMEORIGIN

Pragma: no-cache

Content-Length: length_of_body (actual length of HTTP message body)

Cache-Control: max-age=0

Expires: Fri, 04 Mar 2022 21:12:57 GMT

Keep-Alive: timeout=30, max=100

```
Connection: Keep-Alive
Content-Type: text/xml

<?xml version="1.0" encoding="UTF-8"?>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/"><S:Body>
    <Response Version="3" xmlns="http://securetransport.dw/rcservice/soap">
        <RespClientID>
                <DID>Sample_DID</DID>
                <ClientRef>Sample_ClientRef</ClientRef>
        </RespClientID>
        <Status StatusCode="OK"></Status>
        <SessionTransactionResponse>
                <ReturnCode>006</ReturnCode>
                <Payload Encoding="cdata"></Payload>
        </SessionTransactionResponse>
    </Response>
</S:Body></S:Envelope>
```

Note: The sample below of TerminateSessionResponse is only provided for your reference and thus contains only sample values.

```
HTTP/1.1 200 OK
Date: Fri, 04 Mar 2022 21:12:57 GMT
Server: The Server returning the Response
Cache-Control: no-cache, no-store, must-revalidate
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
Strict-Transport-Security: max-age=31536000; includeSubdomains;
Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';
X-Frame-Options: SAMEORIGIN
Pragma: no-cache
Content-Length: length_of_body (actual length of HTTP message body)
Cache-Control: max-age=0
Expires: Fri, 04 Mar 2022 21:12:57 GMT
Keep-Alive: timeout=30, max=100
Connection: Keep-Alive
Content-Type: text/xml

<?xml version="1.0" encoding="UTF-8"?>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/"><S:Body>
```

```
    <Response Version="3" xmlns="http://securetransport.dw/rcservice/soap">
        <RespClientID>
                <DID>Sample_DID</DID>
                <ClientRef>Sample_ClientRef</ClientRef>
        </RespClientID>
        <Status StatusCode="OK"></Status>
        <TerminateSessionResponse>
                <ReturnCode>006</ReturnCode>
        </TerminateSessionResponse>
    </Response>
</S:Body>
</S:Envelope>
```

Sample SOAP Error Response without a Return Code

Note: The sample below is only provided for your reference and thus contains only sample values.

```
HTTP/1.1 200 OK
Date: Fri, 04 Mar 2022 21:12:57 GMT
Server: The Server returning the Response
Cache-Control: no-cache, no-store, must-revalidate
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
Strict-Transport-Security: max-age=31536000; includeSubdomains;
Content-Security-Policy: default-src 'none'; script-src 'self' 'strict-dynamic' 'nonce-XXXXXXXXXXXXX' 'unsafe-inline'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'none';
X-Frame-Options: SAMEORIGIN
Pragma: no-cache
Content-Length: length_of_body (actual length of HTTP message body)
Cache-Control: max-age=0
Expires: Fri, 04 Mar 2022 21:12:57 GMT
Keep-Alive: timeout=30, max=100
Connection: Keep-Alive
Content-Type: text/xml

<?xml version="1.0" encoding="UTF-8"?>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
<S:Body>
    <Response Version="3" xmlns="http://securetransport.dw/rcservice/soap">
        <RespClientID>
                <DID>Sample_DID</DID>
```

```
                <ClientRef>Sample_ClientRef</ClientRef>
            </RespClientID>
            <Status StatusCode="XMLError">Request missing data: &lt;DID&gt;</Status>
        </Response>
</S:Body>
</S:Envelope>
```

Error Management

In this Chapter

- Error Handling
- Error Types
- HTTP Errors & Handling
- Return Code Values
- Status Code Values

Error Handling

If the function succeeds, the immediate return value is zero. If the function fails, the immediate return value is a non-zero error code listed in the "Return Code Values" section. Please use the retruned error code to determine:

- The error's meaning.
- If the transaction must be retried.

Error Types

An application using the Datawire Secure Transport XML and SOAP Interface may receive HTTP Errors or Datawire Secure Transport Network Errors reported by the Datawire Secure Transport Network, which include **StatusCode** and **ReturnCode** Errors. For more information, see the sections below.

HTTP Errors & Handling

The following table describes two potential HTTP return error scenarios and what to do if you receive these errors. Retry column only applies to Simple Transactions and Initiate Session Transactions. If an HTTP error occurs during session transaction, this session should be terminated and a new session started.

Returned Error	Error Text	Origin	Processor Received?	Timeout Reversal?	Retry?	Description
HTTP 4XX	NA	DW[a]	No	No	No	Invalid Request
HTTP 5XX	NA	DW	Unknown	Yes	Yes	System Internal Error.

[a] DW is the short form for "Datawire Secure Transport".

Exceptions

Condition	Processor Received?	Timeout Reversal?	Retry?
Connection broken prematurely (TCP/TLS Connection failure)	Unknown	Yes	Yes
No transaction response received within recommended platform timeout value	Unknown	Yes (Depending on the transaction request type, it may not be necessary to perform a Timeout Reversal. Confirm with your host certification analyst for more information.)	Yes

Return Code Values

Retry column in the below table only applies to Simple and Initiate Session transactions. If an error occurs during the session transaction, this session should be terminated and a new session started.

A successful response will return the **ReturnCode** "000". When a **ReturnCode** is not "000", it can be one of the **ReturnCodes** listed below.

If the transaction is unsuccessful, there may not be a **ReturnCode** value. In this case, the **StatusCode** will contain the error details, which will help manage the error.

Important: Any error incurred mid session is generally not recoverable. The session should be restarted, if an error is encountered mid session.

return code	error text	origin	processor received?	timeout reversal?	retry ?	description
006	Invalid session	DW[a] Host	No	No	No	The session context provided in the request is not valid or has expired.
200	Host Busy	DW Host	No	No	Yes	The processor's Host is busy and is currently unable to service this request.
202	Host Connect Error	DW Host	No	No	Yes	Could not connect to the processor's Host.
203	Host Drop	DW Host	Yes	Yes	Yes	The processor's Host disconnected during the transaction before sending a response.
204	Host Comm Error	DW Host	Unknown	Yes	No	An error was encountered while communicating with the processor's Host.
205	No Response	DW Host	Yes	Yes	Yes	No response from the processor's Host
206	Host Send Error	DW Host	Unknown	Yes	Yes	An error was encountered when sending the request to the processor and the Host daemon cannot continue sending packets to the processor because the connection is broken.

[a] DW is the short form for "Datawire Secure Transport".

return code	error text	origin	processor received?	timeout reversal?	retry ?	description
405	Datawire Secure Transport Network Timeout	DW Host	Unknown	Yes	Yes	The request could not be processed.
505	Network Error	DW Host	Unknown	Yes	Yes	The request could not be processed.
008	Network Error	DW Host	Unknown	Yes	Yes	The request could not be processed.

Status Code Values

Each transaction Response contains a **StatusCode**. When a transaction is successfully passed from the application, through the Datawire Secure Transport Network, the response's **StatusCode** reads "**OK**" and the **ReturnCode** reads "**000**". Even though the transaction has successfully passed through the Datawire Secure Transport Network, it can still fail to be processed successfully at the payment processor.

If a request is not successfully passed through the Datawire Secure Transport Network, the **StatusCode** will be represented by one of the codes listed in the **StatusCode** section below.

The Status Element

The Status element is included in every response. The Status element describes the result of the request. The StatusCode sub-element contains the result code of the request and a description of the result code.

Status Code	Status Text	Origin	Processor Received?	Timeout Reversal	Retry?
OK	Even though the transaction has successfully passed through the Datawire Secure Transport Network, it can still fail to be processed successfully at the payment processor.	DW Host	Unknown	Refer to Return Code Values table	No
AuthenticationError	DID in the request was not successfully authenticated.	DW Host	No	No	No
UnknownServiceID	ServiceID is unknown.	DW Host	No	No	No
XMLError	Request contains some XML error, such as malformed XML, violation of this DTD, etc.	DW Host	No	No	No
OtherError	Returned when the transaction could not be processed by one of the Datawire Secure Transport Servers.	DW Host	No	No	No
WrongSessionContext	The SessionContext element of the Session Transaction and/or Terminate Session transaction request(s) does not match the SessionContext returned by the InitiateSession response.	DW Host	No	No	No
Timeout	Invalid ClientTimeout value in the Initiate Session or Terminate Session request (e.g. timeout value is too small).	DW Host	No	No	No

SOAP Interface Code Samples

For more information regarding the SOAP components, refer to the WSDL file.

C# Sample

In this sample, we are assuming that the Reference Name is "**RCServiceRef1**".

Warning: The sample code is for your reference. This sample is not suitable for direct production use.

```
// IMPORTANT: This sample is for demonstration purpose only.
//It is not suitable for direct production use.

RCServiceRef1.rcPortTypeClient client = new RCServiceRef1.rcPortTypeClient(); RCServiceRef1.ReqClientIDType reqID;
RCServiceRef1.RequestType req;

// Simple
reqID = new RCServiceRef1.ReqClientIDType();
reqID.App = "Sample_App";
reqID.Auth = "Sample_AuthKey1|Sample_AuthKey2";
reqID.ClientRef = "Sample_CRef00001";
reqID.DID = "Sample_Did003";

req = new RCServiceRef1.RequestType();
req.ClientTimeout = "Sample_ClientTimeout";
req.ReqClientID = reqID;
req.Version = "3";

RCServiceRef1.TransactionType tran = new RCServiceRef1.TransactionType();
tran.ServiceID = "Sample_ServiceID";
tran.Payload = new RCServiceRef1.PayloadType();
tran.Payload.EncodingSpecified = true;
tran.Payload.Encoding = RCServiceRef1.PayloadTypeEncoding.xml_escape;
tran.Payload.Value = "<RC>Hello, A & B</RC>";
```

//Comment: You must only apply the encoding once for the payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly.

```
req.Item = tran;
RCServiceRef1.ResponseType resp = client.rcTransaction(req);

Debug.WriteLine("TransactionResponse Payload:" + ((RCServiceRef1.TransactionResponseType)(resp.Item)).Payload.Value);
```

```
// Initiate Session
reqID = new RCServiceRef1.ReqClientIDType();
reqID.App = "Sample_App";
reqID.Auth = "Sample_AuthKey1|Sample_AuthKey2";
reqID.ClientRef = "Sample_CRef00002";
reqID.DID = "Sample_Did003";

req = new RCServiceRef1.RequestType();
req.ClientTimeout = "Sample_ClientTimeout";
req.ReqClientID = reqID;
req.Version = "3";

RCServiceRef1.InitiateSessionType sessInit = new RCServiceRef1.InitiateSessionType(); sessInit.ServiceID =
"Sample_ServiceID";
sessInit.Payload = new RCServiceRef1.PayloadType();
sessInit.Payload.EncodingSpecified = true;
sessInit.Payload.Encoding = RCServiceRef1.PayloadTypeEncoding.xml_escape;
sessInit.Payload.Value = "<RC>Hello, A & B</RC>";
```

//Comment: You must only apply the encoding once for the payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly.

```
req.Item = sessInit;
resp = client.rcTransaction(req);

// Save the Session Context
RCServiceRef1.InitiateSessionResponseType r = (RCServiceRef1.InitiateSessionResponseType)resp.Item;
String sessContext = r.SessionContext;

Debug.WriteLine("InitiateSessionResponse Payload:" +
((RCServiceRef1.InitiateSessionResponseType)(resp.Item)).Payload.Value);

// Session Transaction
reqID = new RCServiceRef1.ReqClientIDType();
reqID.App = "Sample_App";
reqID.Auth = "Sample_AuthKey1|Sample_AuthKey2";
reqID.ClientRef = "Sample_CRef00003";
reqID.DID = "Sample_Did003";

req = new RCServiceRef1.RequestType();
```

```
req.ClientTimeout = "Sample_ClientTimeout";
req.ReqClientID = reqID;
req.Version = "3";

RCServiceRef1.SessionTransactionType sessTx = new RCServiceRef1.SessionTransactionType();
sessTx.SessionContext = sessContext;
sessTx.Payload = new RCServiceRef1.PayloadType();
sessTx.Payload.EncodingSpecified = true;
sessTx.Payload.Encoding = RCServiceRef1.PayloadTypeEncoding.xml_escape;
sessTx.Payload.Value = "<RC>Hello, A & B</RC>";
```

//Comment: You must only apply the encoding once for the payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly.

```
req.Item = sessTx;
resp = client.rcTransaction(req);

Debug.WriteLine("SessionTransactionResponse Payload:" +
((RCServiceRef1.SessionTransactionResponseType)(resp.Item)).Payload.Value);

// Terminate Session
reqID = new RCServiceRef1.ReqClientIDType();
reqID.App = "Sample_App";
reqID.Auth = "Sample_AuthKey1|Sample_AuthKey2";
reqID.ClientRef = "Sample_CRef00004";
reqID.DID = "Sample_Did003";

req = new RCServiceRef1.RequestType();
req.ClientTimeout = "Sample_ClientTimeout";
req.ReqClientID = reqID;
req.Version = "3";

RCServiceRef1.TerminateSessionType sessTerm = new RCServiceRef1.TerminateSessionType();
sessTerm.SessionContext = sessContext;
sessTerm.Payload = new RCServiceRef1.PayloadType();
sessTerm.Payload.EncodingSpecified = true;
sessTerm.Payload.Encoding = RCServiceRef1.PayloadTypeEncoding.xml_escape;
sessTerm.Payload.Value = "<RC>Hello, A & B</RC>";
```

//Comment: You must only apply the encoding once for the payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it

explicitly.

```
req.Item = sessTerm;
resp = client.rcTransaction(req);

Debug.WriteLine("TerminateSessionResponse Payload:" +
((RCServiceRef1.TerminateSessionResponseType)(resp.Item)).Payload.Value);
```

VB Net Sample

In this sample, we are assuming that the Reference Name is "**RCServiceRef1**".

Warning: The sample code is for your reference. This sample is not suitable for direct production use.

```
'    IMPORTANT: This sample is for demonstration purpose only.
'    It is not suitable for direct production use.

Dim mySvc As New RCServiceRef1.rcPortTypeClient
Dim objResponse As RCServiceRef1.ResponseType
Dim objRequest As RCServiceRef1.RequestType

'    Simple
objRequest = New RCServiceRef1.RequestType
objRequest.ClientTimeout = "Sample_ClientTimeout"
objRequest.ReqClientID = New RCServiceRef1.ReqClientIDType()
objRequest.ReqClientID.App = "Sample_App"
objRequest.ReqClientID.Auth = "Sample_AuthKey1|Sample_AuthKey2"
objRequest.ReqClientID.ClientRef = "Sample_CRef001"
objRequest.Version = "3"
objRequest.ReqClientID.DID = "Sample_Did003"

Dim reqTxn As New RCServiceRef1.TransactionType()
reqTxn.ServiceID = "Sample_ServiceID"
reqTxn.Payload = New RCServiceRef1.PayloadType()
reqTxn.Payload.EncodingSpecified = True
reqTxn.Payload.Encoding = RCServiceRef1.PayloadTypeEncoding.xml_escape
```

//Comment: You must only apply the encoding once for the payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly.

```
reqTxn.Payload.Value = "<RC>Hello, A & B</RC>"
objRequest.Item = reqTxn
```

// in this sample, it is assumed that the soap library applies the encoding automatically:

```
objResponse = mySvc.rcTransaction(objRequest)

Dim txResp As RCServiceRef1.TransactionResponseType
txResp = CType(objResponse.Item, RCServiceRef1.TransactionResponseType) Debug.WriteLine("TransactionResponse
Payload:" + txResp.Payload.Value)
```

```
'    Session Init
objRequest = New RCServiceRef1.RequestType
objRequest.ClientTimeout = "Sample_ClientTimeout"
objRequest.ReqClientID = New RCServiceRef1.ReqClientIDType()
objRequest.ReqClientID.App = "Sample_App"
objRequest.ReqClientID.Auth = "Sample_AuthKey1|Sample_AuthKey2"
objRequest.ReqClientID.ClientRef = "Sample_CRef002"
objRequest.Version = "3"
objRequest.ReqClientID.DID = "Sample_Did003"

Dim reqSessInit As New RCServiceRef1.InitiateSessionType()
reqSessInit.ServiceID = "Sample_ServiceID"
reqSessInit.Payload = New RCServiceRef1.PayloadType()
reqSessInit.Payload.EncodingSpecified = True
reqSessInit.Payload.Encoding = RCServiceRef1.PayloadTypeEncoding.xml_escape
```

//Comment: You must only apply the encoding once for the payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly.

```
reqSessInit.Payload.Value = "<RC>Hello, A & B</RC>"
objRequest.Item = reqSessInit
```

// in this sample, it is assumed that the soap library applies the encoding automatically:

```
objResponse = mySvc.rcTransaction(objRequest)

Get session context from the response
Dim sessContext As String
Dim initResp As RCServiceRef1.InitiateSessionResponseType

initResp = CType(objResponse.Item, RCServiceRef1.InitiateSessionResponseType) sessContext = initResp.SessionContext

Debug.WriteLine("InitiateSessionResponse Payload:" + initResp.Payload.Value)

'    Session Transaction

objRequest = New RCServiceRef1.RequestType
objRequest.ClientTimeout = "Sample_ClientTimeout"
objRequest.ReqClientID = New RCServiceRef1.ReqClientIDType()
objRequest.ReqClientID.App = "Sample_App"
objRequest.ReqClientID.Auth = "Sample_AuthKey1|Sample_AuthKey2"
```

```
objRequest.ReqClientID.ClientRef = "Sample_CRef003"
objRequest.Version = "3"
objRequest.ReqClientID.DID = "Sample_Did003"

Dim reqSessTxn As New RCServiceRef1.SessionTransactionType()
reqSessTxn.SessionContext = sessContext
reqSessTxn.Payload = New RCServiceRef1.PayloadType()
reqSessTxn.Payload.EncodingSpecified = True
reqSessTxn.Payload.Encoding = RCServiceRef1.PayloadTypeEncoding.xml_escape
```

//Comment: You must only apply the encoding once for the payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly.

```
reqSessTxn.Payload.Value = "<RC>Hello, A & B</RC>"
objRequest.Item = reqSessTxn
```

// in this sample, it is assumed that the soap library applies the encoding automatically:

```
objResponse = mySvc.rcTransaction(objRequest)

Dim sessResp As RCServiceRef1.SessionTransactionResponseType
sessResp = CType(objResponse.Item, RCServiceRef1.SessionTransactionResponseType)
Debug.WriteLine("SessionTransactionResponse Payload:" + sessResp.Payload.Value)

'    Terminate Session
objRequest = New RCServiceRef1.RequestType
objRequest.ClientTimeout = "Sample_ClientTimeout"
objRequest.ReqClientID = New RCServiceRef1.ReqClientIDType()
objRequest.ReqClientID.App = "Sample_App"
objRequest.ReqClientID.Auth = "Sample_AuthKey1|Sample_AuthKey2"
objRequest.ReqClientID.ClientRef = "Sample_CRef004"
objRequest.Version = "3"
objRequest.ReqClientID.DID = "Sample_Did003"

Dim reqSessTerm As New RCServiceRef1.TerminateSessionType()
reqSessTerm.SessionContext = sessContext
reqSessTerm.Payload = New RCServiceRef1.PayloadType()
reqSessTerm.Payload.EncodingSpecified = True
reqSessTerm.Payload.Encoding = RCServiceRef1.PayloadTypeEncoding.xml_escape
```

//Comment: You must only apply the encoding once for the payload value, which is the raw Rapid Connect XML

message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly.

```
reqSessTerm.Payload.Value = "<RC>Hello, A & B</RC>"

objRequest.Item = reqSessTerm
```

// in this sample, it is assumed that the soap library applies the encoding automatically:

```
objResponse = mySvc.rcTransaction(objRequest)

Dim termResp As RCServiceRef1.TerminateSessionResponseType
termResp = CType(objResponse.Item, RCServiceRef1.TerminateSessionResponseType)
Debug.WriteLine("TerminateSessionResponse Payload:" + termResp.Payload.Value)
```

JAVA Sample

Warning: The sample code is for your reference. This sample is not suitable for direct production use.

```
//IMPORTANT: This sample is for demonstration purpose only.
//It is not suitable for direct production use. RcService rcService = new RcService();
RcPortType rcPort = rcService.getRcServicePort();

ResponseType response;
RequestType request;
ReqClientIDType reqClientID;
PayloadType payload;

//Simple transaction
request = new RequestType();
request.setClientTimeout(new BigInteger("Sample_ClientTimeout"));
request.setVersion("3");

reqClientID = new ReqClientIDType();
reqClientID.setApp("Sample_App");
reqClientID.setAuth("Sample_AuthKey1|Sample_AuthKey2");
reqClientID.setClientRef("Sample_CRef001");
reqClientID.setDID("Sample_Did003");

request.setReqClientID(reqClientID);

payload = new PayloadType();
payload.setEncoding("xml_escape");
payload.setValue("<RC>Hello, A & B</RC>");
```

//Comment: You must only apply the encoding once for the payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly.

```
TransactionType transaction = new TransactionType();
transaction.setServiceID("Sample_ServiceID");
transaction.setPayload(payload);
request.setTransaction(transaction);

response = rcPort.rcTransaction(request);

System.out.println("TransactionResponse Payload: " + response.getTransactionResponse().getPayload().getValue());
```

```
//InitiateSession
request = new RequestType();
request.setClientTimeout(new BigInteger("Sample_ClientTimeout"));
request.setVersion("3");

reqClientID = new ReqClientIDType();
reqClientID.setApp("Sample_App");
reqClientID.setAuth("Sample_AuthKey1|Sample_AuthKey2");
reqClientID.setClientRef("Sample_CRef002");
reqClientID.setDID("Sample_Did003");

request.setReqClientID(reqClientID);

payload = new PayloadType();
payload.setEncoding("xml_escape");
payload.setValue("<RC>Hello, A & B</RC>");
```

//Comment: You must only apply the encoding once for the payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly.

```
InitiateSessionType initiateSession = new InitiateSessionType(); initiateSession.setServiceID("Sample_ServiceID");
initiateSession.setPayload(payload);

request.setInitiateSession(initiateSession);

response = rcPort.rcTransaction(request);
System.out.println("InitiateSessionResponse Payload: " + response.getInitiateSessionResponse().getPayload().getValue());

String sessionContext = response.getInitiateSessionResponse().getSessionContext();

//SessionTransaction
request = new RequestType();
request.setClientTimeout(new BigInteger("Sample_ClientTimeout"));
request.setVersion("3");

reqClientID = new ReqClientIDType();
reqClientID.setApp("Sample_App");
reqClientID.setAuth("Sample_AuthKey1|Sample_AuthKey2");
reqClientID.setClientRef("Sample_CRef003");
```

```
reqClientID.setDID("Sample_Did003");

request.setReqClientID(reqClientID);

payload = new PayloadType();
payload.setEncoding("xml_escape");
payload.setValue("<RC>Hello, A & B</RC>");
```

//Comment: You must only apply the encoding once for the payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly.

```
SessionTransactionType sessionTransaction = new SessionTransactionType();
sessionTransaction.setSessionContext(sessionContext);
sessionTransaction.setPayload(payload);

request.setSessionTransaction(sessionTransaction);

response = rcPort.rcTransaction(request);
System.out.println("SessionTransactionResponse Payload: " +
response.getSessionTransactionResponse().getPayload().getValue());

//TerminateSession
request = new RequestType();
request.setClientTimeout(new BigInteger("Sample_ClientTimeout"));
request.setVersion("3");

reqClientID = new ReqClientIDType();
reqClientID.setApp("Sample_App");
reqClientID.setAuth("Sample_AuthKey1|Sample_AuthKey2");
reqClientID.setClientRef("Sample_CRef004");
reqClientID.setDID("Sample_Did003");

request.setReqClientID(reqClientID);

payload = new PayloadType();
payload.setEncoding("xml_escape");
payload.setValue("<RC>Hello, A & B</RC>");
```

//Comment: You must only apply the encoding once for the payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly.

```
TerminateSessionType terminateSession = new TerminateSessionType();
terminateSession.setSessionContext(sessionContext);
terminateSession.setPayload(payload);

request.setTerminateSession(terminateSession);

response = rcPort.rcTransaction(request);

System.out.println("TerminateSessionResponse Payload: " +
response.getTerminateSessionResponse().getPayload().getValue());
```

PHP Sample

Warning: The sample code is for your reference. This sample is not suitable for direct production use.

```
# IMPORTANT: This sample is for demonstration purpose only.
#It is not suitable for direct production use.

$wsdl = "https://path/to/rcservice.wsdl";

$client = new SoapClient($wsdl, array("trace"=>1, "exceptions"=>1));

class ReqClientID {
	function ReqClientID($did, $app, $auth, $clientRef)
		{
			$this->DID = $did;
			$this->App = $app;
			$this->Auth = $auth;
			$this->ClientRef = $clientRef;
		}
}

class PayloadXml {
	function PayloadXml($payload)
		{
			$this->Encoding = "xml_escape";
			$this->_ = $payload;
		}
}

class Transaction {
	function Transaction($serviceID, $payload)
		{
			$this->ServiceID = $serviceID;
			$this->Payload = $payload;
		}
}

class InitiateSession {
	function InitiateSession($serviceID, $payload)
		{
```

```
                $this->ServiceID = $serviceID;
                $this->Payload = $payload;
        }
}

class SessionTransaction {
    function SessionTransaction($sessionContext, $payload)
        {
                $this->SessionContext = $sessionContext;
                $this->Payload = $payload;
        }
}

class TerminateSession {
    function TerminateSession($sessionContext, $payload)
        {
                $this->SessionContext = $sessionContext;
                $this->Payload = $payload;
        }
}

### Simple Transaction
$reqClientID = new ReqClientID("Sample_Did003", "Sample_App",
"Sample_AuthKey1|Sample_AuthKey2", "Sample_CRef00001");
$payld = new PayloadXml("<RC>Hello, A & B.</RC>");
```

//Comment: You must only apply the encoding once for the payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly.

```
$tranx = new Transaction("Sample_ServiceID", $payld);

$parameters= array(
                "Version" => "3",
                "ClientTimeout" => "Sample_ClientTimeout",
                "ReqClientID" => $reqClientID,
                "Transaction" => $tranx);

$response = $client->rcTransaction($parameters);
```

```
print "<br />\n TransactionResponse payload: ".htmlspecialchars($response-
>TransactionResponse->Payload->_);

### Initiate Session
$reqClientID = new ReqClientID("Sample_Did003",
"Sample_App", "Sample_AuthKey1|Sample_AuthKey2", "Sample_CRef00002");

$payld = new PayloadXml("<RC>Hello, A & B.</RC>");
```

//Comment: You must only apply the encoding once for the payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly.

```
$initSession = new InitiateSession("Sample_ServiceID", $payld);

$parameters= array(
                    "Version" => "3",
                    "ClientTimeout" => "Sample_ClientTimeout",
                    "ReqClientID" => $reqClientID,
                    "InitiateSession" => $initSession);

$response = $client->rcTransaction($parameters);

# Retrieve the session context and use in the following session transactions.
$sessCxt = $response->InitiateSessionResponse->SessionContext;

print "<br />\n InitiateSessionResponse payload: ".htmlspecialchars($response-
>InitiateSessionResponse->Payload->_);

### Session Transaction
$reqClientID = new ReqClientID("Sample_Did003",
"Sample_App", "Sample_AuthKey1|Sample_AuthKey2", "Sample_CRef00003");

$payld = new PayloadXml("<RC>Hello, A & B.</RC>");
```

//Comment: You must only apply the encoding once for the payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly.

```
$sessTxn = new SessionTransaction($sessCxt, $payld);

$parameters= array(
```

```
                    "Version" => "3",
                    "ClientTimeout" => "Sample_ClientTimeout",
                    "ReqClientID" => $reqClientID,
                    "SessionTransaction" => $sessTxn);

$response = $client->rcTransaction($parameters);

print "<br />\n SessionTransactionResponse payload: ".htmlspecialchars($response-
>SessionTransactionResponse->Payload->_);

### Terminate Session
$reqClientID = new ReqClientID("Sample_Did003",
"Sample_App", "Sample_AuthKey1|Sample_AuthKey2", "Sample_CRef00004");

$payld = new PayloadXml("<RC>Hello, A & B.</RC>");
```

//Comment: You must only apply the encoding once for the payload value, which is the raw Rapid Connect XML message: this can be done automatically by your soap library or using your own code and application to invoke it explicitly.

```
$termSession = new TerminateSession($sessCxt, $payld);

$parameters= array(
                    "Version" => "3",
                    "ClientTimeout" => "Sample_ClientTimeout ",
                    "ReqClientID" => $reqClientID,
                    "TerminateSession" => $termSession);

$response = $client->rcTransaction($parameters);

print "<br />\n TerminateSessionResponse payload: ".htmlspecialchars($response-
>TerminateSessionResponse->Payload->_);
```

Connect With Us

For more information about Datawire:

securetransport.integration@fiserv.com

fiserv.com

Fiserv is driving innovation in Payments, Processing Services, Risk & Compliance, Customer & Channel Management and Insights & Optimization. Our solutions help clients deliver financial services at the speed of life to enhance the way people live and work today.

Visit fiserv.com to learn more.

Fiserv, Inc.
255 Fiserv Drive
Brookfield, WI 53045
fiserv.com

© 2025 Fiserv, Inc. or its affiliates. Fiserv is a trademark of Fiserv, Inc., registered or used in the United States and foreign countries and may or may not be registered in your country. All trademarks, service marks and trade names referenced in this material are the property of their respective owners. 870731 01/22

fiserv.